

Gateway OS / OpenWrt — Operator Manual

What this technology does

Gateway-01 is a Raspberry Pi 4B running a customized ChirpStack Gateway OS 4.12.0 / OpenWrt-class image. The OS hosts the RAK5146 radio services, MQTT Forwarder, local Mosquitto buffer, gateway evidence runtime, and SIM7600 LTE networking.

This layer is the operating platform. A booted gateway does **not** by itself prove RF, MQTT, LTE, or application delivery.

Current commissioned boundary

- Platform: Raspberry Pi 4B.
- Gateway OS family: ChirpStack Gateway OS 4.12.0-derived custom image.
- Accepted radio: RAK5146/SX1303.
- Region: **AS923**.
- Gateway EUI: 0016c001f139a1cb.
- Production Internet path: SIM7600 QMI interface `wwan0`.
- Ethernet: management-only.
- Wi-Fi: fallback path.
- Reflash only for a real regression, media failure, or intentional image change.

Important paths

Purpose	Path
OpenWrt/UCI configuration	<code>/etc/config/</code>
Mosquitto configuration	<code>/etc/mosquitto/</code>
Gateway image overlay source in Git	<code>deployment/gateway/image-overlay/</code>
Runtime logs	<code>logread</code> / service-specific logs
Gateway evidence state	use the paths defined by the evidence runbook; preserve it during ordinary recovery

Never assume `/tmp` is persistent. Do not store the only copy of a certificate, recovery archive, or research evidence there.

Step 1 — Confirm you are on Gateway-01

Run on the gateway shell:

```
cat /etc/os-release
uname -a
```

```
ubus call system board
date -u
```

PASS means: the board is the expected Raspberry Pi/Gateway OS environment and UTC is plausible.

If this fails: do not troubleshoot ChirpStack or MQTT yet. Check power, SD card boot, console, or management networking.

Step 2 — Check storage before doing anything else

```
df -h
mount
```

PASS means: root/overlay storage is writable and has free space. A nearly full filesystem can make otherwise unrelated services fail.

If free space is dangerously low: do not blindly delete /etc, certificates, journal state, Mosquitto persistence, or evidence files. Use the backup/recovery manual to identify safe cleanup targets.

Step 3 — Check interfaces and routes

```
ip -br addr 2>/dev/null || ip addr
ip route
ubus call network.interface dump
```

PASS means: management access is present and the intended production path can be identified. On the commissioned gateway, normal production Internet should converge to `wwan0`; Ethernet must remain management-only.

Do not add a manual default route merely because LTE is still converging immediately after boot. The commissioned `lte-route-health` controller owns production route selection.

Step 4 — Check failed/restarting services

```
monit status 2>/dev/null || true
ps w | grep -E '[c]hirpstack-concentrator|[c]hirpstack-mqtt-forwarder|[m]osquitto|[l]te-route-health'
logread | tail -n 120
```

PASS means: expected services are present and there is no obvious crash loop, storage error, kernel fault, or repeated modem reset.

A service showing as running is only a process-level check. Continue to the radio, MQTT, and LTE guides for protocol-level proof.

Step 5 — Inspect configuration safely

Use UCI instead of dumping unrelated secret-bearing files:

```
uci show network
uci show chirpstack-concentrator
```

```
uci show chirpstack-mqtt-forwarder
uci show chirpstack-udp-forwarder
```

Expected high-level facts:

- Concentratord uses the SX1302/SX1303 RAK5146 profile.
- region/channel plan is AS923.
- MQTT Forwarder points to tcp://127.0.0.1:1883.
- MQTT topic prefix is as923.
- legacy UDP forwarding is disabled.

Do not copy an old inactive SX1301 Gateway EUI into current configuration.

Step 6 — Back up before any OS/configuration mutation

For a small UCI/configuration change, first create a Gateway OS backup using **LuCI -> System -> Backup / Flash Firmware -> Generate archive** and copy it off the gateway.

For image/reflash work, also preserve the protected MQTT/evidence identity material according to [../gateway/operations/02-backup-and-recovery.md](#). A normal sysupgrade archive is not guaranteed to contain every private identity or persistent data file.

Hard stop: do not reflash until an off-gateway rollback copy and a known-good image are available.

Step 7 — Make changes with the smallest possible blast radius

For UCI-managed settings:

1. inspect the current section with `uci show`;
2. change only the required option through LuCI/UCI;
3. save/commit;
4. restart or reload only the affected service;
5. immediately run that service's verification steps.

Do not reboot the entire gateway merely to apply a single service change unless that technology specifically requires reboot-level validation.

Step 8 — Post-change platform verification

```
ubus call system board
date -u
df -h
ip route
monit status 2>/dev/null || true
logread | tail -n 100
```

Then verify the affected functional layer:

- radio change -> [02-rak5146-concentratord-as923.md](#)
- LTE/network change -> [03-sim7600-lte-backhaul.md](#)
- MQTT change -> [04-mosquitto-mqtt.md](#)

Troubleshooting order

Symptom	First layer to inspect
Gateway completely unreachable	power, SD boot, management interface
Shell works but RAK5146 unavailable	Concentrator/SPI/RF guide
RF works but cloud delivery fails	local Mosquitto then LTE
LTE works but MQTT bridge fails	TLS identity/ACL/cloud MQTT
Cloud MQTT works but no app event	ChirpStack
Application works but evidence missing	gateway evidence services

Recovery rules

- Preserve identities before reinstalling.
- Preserve current journal/evidence state unless the recovery procedure explicitly calls for a deliberate lineage reset.
- A new SD card/image is not automatically a new Gateway EUI; the active RAK5146 identity remains the radio authority.
- After restore/reflash, verify time, storage, RAK5146 EUI, AS923, local MQTT, LTE routing, cloud MQTT, and evidence path before calling recovery complete.

Completion checklist

The OS layer is healthy when all are true:

- correct Gateway OS/platform is booted;
- UTC is correct;
- persistent storage has safe free space;
- expected interfaces exist;
- production routing can converge correctly;
- no immediate crash loop or failed core service exists;
- effective UCI configuration still matches the current architecture;
- the next functional-layer checks pass.

Detailed rebuild/recovery runbooks

- [../gateway/setup/02-install-chirpstack-gateway-os.md](#)
- [../gateway/setup/02a-build-sim7600-capable-gateway-os.md](#)
- [../gateway/setup/06-verify-gateway-os.md](#)
- [../gateway/operations/02-backup-and-recovery.md](#)
- [../gateway/operations/05-troubleshooting.md](#)

Supporting runbook: 02-backup-and-recovery.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Operations 2. Gateway Backup and Recovery

A recoverable gateway needs **four different recovery sets** because none can replace the others:

1. the exact Gateway OS Base factory image, which can boot a blank SD card;
2. an encrypted Gateway OS configuration archive, which restores UCI and service settings;
3. an encrypted MQTT/evidence-upload identity bundle, which restores the machine identities needed to reach the server; and
4. the gateway-integrity continuity record: reviewed journal implementation/version/hash, journal-format version, last valid local sequence/record/segment hash, latest accepted server checkpoint/receipt, and any closed unuploaded segments required by the retention policy.

Keep these outside the gateway. Test them on spare media when available. If only one SD card exists, do not treat the missing spare as an automatic blocker; instead require a reflash-ready rollback boundary before overwriting the card: retain the exact approved factory image and SHA-256, keep the verified configuration and protected identity bundles off-gateway, confirm an SD writer is available, and plan for downtime while the same card is reflashed if rollback is required.

[!IMPORTANT] A journal restore is not ordinary file restoration. If the server already holds a checkpoint newer than the local backup, restoring the old journal and continuing from it would be a rollback conflict. Preserve the server anchor and use the reviewed recovery/epoch procedure; never reset the local sequence or delete the newer cloud checkpoint to make an old SD image look current.

Step 1: Inspect the running gateway before backup

```
cat /etc/os-release
uname -a
monit status
uci show chirpstack-concentrator
uci show chirpstack-mqtt-forwarder
uci show chirpstack-udp-forwarder
opkg list-installed | grep '^mosquitto'
sha256sum /etc/mosquitto/mosquitto.conf
ls -l /etc/mosquitto/data /etc/mosquitto/certs
```

Keep the queue limits, current database size, free-space reserve, certificate fingerprint/expiry, management address, and last successful drain test with the backup location. When the journal is enabled, also keep:

```
journal implementation/version + executable SHA-256
journal format version
journal storage budget + path
last valid local sequence + record hash
last closed segment ID + segment hash
latest accepted server checkpoint ID/sequence/record hash/segment hash
```

```
unuploaded closed-segment inventory
last successful journal upload/reconciliation test
```

These values let the restored gateway be compared with the working one. Do not print or copy private-key contents or unrestricted journal payloads into notes.

Step 2: Drain delivery work and anchor journal state before a planned backup

When WAN and the remote broker are healthy:

1. verify the outgoing Mosquitto queue has drained;
2. close/upload all journal segments that can be closed under the configured policy;
3. verify the server accepted those segment digests;
4. create or confirm a final healthy checkpoint that covers the latest closed evidence boundary;
5. record the checkpoint receipt with the backup manifest.

Do not delete `mosquitto.db` to make it appear empty. Do not truncate/rewrite the journal to make it appear synchronized. Confirm queue drain using gateway logs, remote broker events, ChirpStack frame counters, and application deduplication records; confirm journal synchronization through the server verifier/checkpoint store.

Step 3: Export Gateway OS configuration

In LuCI:

```
System > Backup / Flash Firmware > Generate archive
```

When CLI backup is supported by the pinned image, inspect help first:

```
sysupgrade --help
sysupgrade -b /tmp/gateway-os-backup-<DATE>.tar.gz
sha256sum /tmp/gateway-os-backup-<DATE>.tar.gz
```

The archive may contain `/etc/mosquitto/certs/<GATEWAY_EUI>.key` and queued uplinks. Transfer it through a protected path and encrypt it at rest.

Inspect the archive securely and verify it includes:

```
/etc/mosquitto/mosquitto.conf
/etc/mosquitto/certs/
/etc/mosquitto/data/ when queue-state recovery is required
/etc/config/chirpstack-concentrator
/etc/config/chirpstack-mqtt-forwarder
/etc/config/chirpstack-udp-forwarder
/etc/gateway-integrity/config/ when that path is used by the reviewed implementation
/etc/gateway-integrity/state/ continuity metadata required by the reviewed restore procedure
```

Do not assume a generic Gateway OS archive is the only copy of closed journal evidence. If the retention policy requires unuploaded closed segments to survive gateway replacement, preserve them in a separately encrypted evidence backup and keep their object SHA-256/segment hashes in the manifest.

Step 4: Preserve the factory image

Keep the official source, release, filename, calculated SHA-256, and protected storage location for the exact Raspberry Pi 4B Gateway OS Base image used by this gateway. The hash detects accidental changes to the retained artifact, while the source and release identify what should be downloaded again.

A configuration archive is not bootable. If the image file or hash cannot be matched to the tested release, obtain a fresh official image before relying on the recovery plan.

Current Phase 11 single-card evidence - 2026-08-27

The retained Gateway OS 4.12.0 Base factory artifact currently used for rollback preparation is:

```
C:\Users\smartagriintern\lorawan-recovery\gateway-01\factory-v4.12.0\chirpstack-gateway-os-4.12.0-base-  
bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz  
size: 27606919 bytes  
SHA-256: 395e79fe041c4118e10dd4cf796aa426a565d5e733144485d8d014a8d8dbf0a6
```

The two existing off-gateway gateway backup archives were re-hashed successfully, balenaEtcher was detected as the imaging application, a post-flash checklist was preserved beside the factory image, and the live gateway SD card was not touched. This is strong rollback preparation evidence, but the first write to the only production card still requires confirmation that the physical SD reader/writer path is usable and that the maintenance-window downtime is accepted.

Step 5: Preserve the MQTT identity bundle

Keep a separate encrypted recovery bundle containing:

```
Gateway ID  
remote broker CA certificate  
gateway MQTT certificate and private key  
MQTT certificate serial, fingerprint, expiry, and replacement reference  
broker FQDN and port  
region topic prefix  
bridge client IDs  
queue limits and storage path  
  
evidence ingest FQDN when enabled  
evidence-upload CA/client certificate/private key or protected credential reference  
evidence identity fingerprint/expiry/revocation reference  
journal implementation/version/hash and format version  
latest accepted checkpoint/receipt reference  
  
renewal and revocation procedure
```

Never store an unencrypted private key in Git or a general file share.

Step 6: Restore to spare media or the single production card

Preferred path: use a spare SD card so the current working card remains untouched.

Single-card path: when no spare card exists, perform this only during a planned maintenance window after the exact approved factory image, its SHA-256, the verified configuration archive, the protected identity bundle, and a working SD-card writer are all present off-gateway. A failed custom image then requires removing the same card, reflashing the approved factory image, restoring the protected backups, and re-verifying service before the gateway returns to production.

1. Flash the exact approved Gateway OS Base image to the spare card, or to the single production card during the controlled rollback procedure.
2. Boot on an isolated commissioning network.
3. Change the default password.
4. Restore the encrypted configuration archive.
5. Verify local Mosquitto configuration and private-key permissions.
6. Verify the queue path is persistent and not tmpfs.
7. Restore/install the exact reviewed journal implementation and verify its executable hash, format version, evidence path, permissions, and read-only Concentrator source.
8. Compare the restored local journal boundary with the **server's latest accepted checkpoint before starting new evidence writes**.
9. If the restored boundary exactly continues the accepted anchor, follow the normal continuation procedure. If the backup is older than the accepted server anchor, stop and follow the reviewed recovery/new-evidence-epoch procedure; do not rewind the server.
10. Verify MQTT Forwarder still uses `tcp://127.0.0.1:1883` at QoS 1.
11. Verify the two Mosquitto bridge connections use the approved remote endpoint and certificate.
12. Verify the journal uploader uses the approved evidence endpoint/identity and cannot reset sequence/history.
13. Keep UDP Forwarder disabled.
14. Run a fresh WAN-outage, reboot, queue drain, journal upload/reconciliation, OTAA, uplink, and safe downlink test.

Do not overwrite the production SD card during the first restore rehearsal when spare media is available. When only one card exists, there is no non-destructive rehearsal; compensate by completing the reflash-ready rollback gate before the first custom-image write and explicitly accept the additional downtime/recovery risk.

Recovery acceptance

Pass only when the restored gateway preserves the intended Gateway EUI and confirmed RF plan, authenticates with the expected identities, buffers real uplinks across WAN loss and reboot, drains them without duplicate application rows, preserves or explicitly re-establishes the approved journal/checkpoint continuity boundary, uploads/reconciles new evidence correctly, and does not replay a stale downlink. Any unexplained difference in EUI, region, queue path, journal format/sequence/anchor, or certificate identity means the restore is incomplete.

Supporting runbook: 02-install-chirpstack-gateway-os.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 2. Install ChirpStack Gateway OS Base

This procedure installs ChirpStack Gateway OS **Base** on the Raspberry Pi 4B, opens the management interface, sets the initial password, and verifies that the correct image is running.

Gateway OS Base contains the lightweight gateway runtime needed for Concentrator and MQTT Forwarder with an external server. The image may also contain UDP Forwarder, but this repository keeps it disabled. The software integrity journal is an additional reviewed service added later in [Gateway 4A](#); do not assume the base image already provides that custom evidence function. Gateway OS Full adds a local ChirpStack server and is not used in this repository.

Step 1: Download the correct image

What this step does

Obtains the official ChirpStack Gateway OS Base firmware image (.img.gz or .wic.gz) matching the Raspberry Pi 4B hardware architecture and system deployment model from the official release page.

Why we do it

- **Base vs. Full Variant:** Gateway OS Base provides the lightweight radio/forwarding foundation (concentrator, mqtt-forwarder) required when linking to an external Network Server. This repository later adds local Mosquitto buffering and a small reviewed integrity-journal service while still keeping all heavy ChirpStack/application/database/KMS/Fabric roles off the gateway. The Full variant runs a local ChirpStack server stack and is unnecessary for this dedicated gateway node.
- **Hardware Architecture Compatibility:** Raspberry Pi models use different processor architectures (ARMv7 vs. ARMv8/Cortex-A72) and device tree blobs (DTBs). Flashing an image built for a different model will result in a non-bootable system or kernel panic.
- **Factory vs. Sysupgrade Image:** Factory images include raw partition tables, boot sectors, and bootloaders for initial microSD card flashing. Sysupgrade images only replace existing system partitions on an already running OpenWrt instance and cannot boot a blank card.

Procedure

On the administration workstation, open the official ChirpStack Gateway OS Raspberry Pi download page.

Select:

Target: Raspberry Pi 4B
Image type: Base

Artifact: SD card factory image

The downloaded file normally ends in `.wic.gz` or `.img.gz`.

Keep the exact release and filename available while installing:

Gateway OS release: `<GATEWAY_OS_VERSION>`
Downloaded image: `<GATEWAY_OS_IMAGE>`

[!CAUTION] Do not use the Raspberry Pi 3, Raspberry Pi 5, Full, or sysupgrade image for a new Raspberry Pi 4B installation.

Step 2: Verify the downloaded file

What this step does

Calculates the SHA256 cryptographic hash of the downloaded image file on your workstation and compares it against the official checksum published by ChirpStack.

Why we do it

- **Detect Download Corruption:** Guarantees bit-for-bit file integrity before writing to physical storage. Flashing a corrupted download leads to cryptic write errors, unbootable SD cards, silent file corruption, or unexpected runtime crashes.
- **Security Verification:** Confirms that the file has not been altered or corrupted during transit over the internet.

Procedure

Run the checksum verification command corresponding to your operating system:

Linux

```
sha256sum <GATEWAY_OS_IMAGE>
```

macOS

```
shasum -a 256 <GATEWAY_OS_IMAGE>
```

Windows PowerShell

```
Get-FileHash -Algorithm SHA256 .\<GATEWAY_OS_IMAGE>
```

Compare the output hash with the official release checksum. If the values differ, delete the file and download it again.

Step 3: Flash the microSD card

What this step does

Writes raw sector blocks (bootloader, system partition tables, read-only SquashFS root filesystem, and OverlayFS user partition) directly to the physical sectors of the microSD card.

Why we do it

- **Raw Sector Disk Writing:** Creates the initial bootable storage layout required by the Raspberry Pi 4B hardware bootloader.
- **Decline RPi OS Customization:** Gateway OS is built on OpenWrt and uses OpenWrt system initialization routines rather than standard Raspberry Pi OS / `systemd` customization scripts. Applying RPi OS customization scripts injects incompatible configuration files that corrupt startup routines.
- **Ignore Windows Format Prompts:** Gateway OS uses native Linux filesystems (`ext4` and `squashfs`). Windows cannot natively read these partitions and falsely reports them as unformatted or corrupted. Accepting a format prompt will permanently erase the freshly flashed Linux installation.

Procedure

You can use either **Raspberry Pi Imager** or **Balena Etcher**:

Raspberry Pi Imager

1. Insert the microSD card into the workstation.
2. Open **Raspberry Pi Imager**.
3. Select **Choose OS > Use custom**.
4. Choose `<GATEWAY_OS_IMAGE>`.
5. Select the correct microSD card under **Storage**.
6. Start the write.
7. Decline Raspberry Pi OS customization when prompted.
8. Wait for both writing and verification to finish.
9. Eject the card safely.

Balena Etcher

1. Select **Flash from file** and choose `<GATEWAY_OS_IMAGE>`.
2. Select the target microSD card.
3. Start the flash and wait for validation to finish.
4. Eject the card safely.

Windows fix when Balena Etcher displays an error

If Balena Etcher displays `(0, h.requestMetadata) is not a function`, uninstall Etcher, install **Balena Etcher 1.18.11**, run as Administrator, and flash the `.img.gz` file directly.

[!WARNING] After flashing, Windows will prompt to format unrecognized partitions. Select **Cancel** for every format prompt.

Step 4: Perform the first boot

What this step does

Connects all required physical hardware (LoRa antenna, microSD card, network cable) and applies power, allowing Gateway OS to complete its initial partition expansion, SSH key generation, and network initialization.

Why we do it

- **Electrical Safety (Disconnect Power First):** Inserting microSD cards or connecting GPIO hat boards while 5V/3.3V power rails are live can cause voltage spikes, short circuits, or electrostatic discharge (ESD) damage.
- **Hardware Protection (Attach LoRa Antenna):** LoRa concentrator boards (e.g. SX1302/SX1303) transmit RF calibration pulses during startup. Operating without an antenna causes 100% of transmitted RF energy to reflect back into the Power Amplifier (high VSWR), generating extreme heat that can permanently burn out the RF chip in seconds.
- **Network Auto-Provisioning:** Connecting Ethernet allows Gateway OS to obtain an IP address via DHCP and synchronize its system clock via NTP immediately upon booting.
- **Filesystem Expansion & First-Boot Setup:** On first boot, Gateway OS expands its read-write OverlayFS partition to fill the microSD card and generates system UUIDs and SSH host keys. OpenWrt often performs an automated reboot during this sequence to remount the resized filesystem cleanly. Removing power during this setup will corrupt the filesystem.

Procedure

1. Disconnect Raspberry Pi power cord.
2. Insert the flashed microSD card firmly into the card slot.
3. Confirm that the LoRa antenna is securely attached to the concentrator board.
4. Connect an Ethernet cable to a local network with DHCP service available.
5. Apply power.
6. Leave the gateway running for at least 3–5 minutes.

[!IMPORTANT] The gateway may reboot automatically during its first startup. Do not remove power while the first boot is still completing.

Step 5: Find the gateway address

What this step does

Locates the IP address assigned to the gateway on your local network (via your router's DHCP lease table) or connects to the gateway's fallback Wi-Fi commissioning Access Point (ChirpStackAP-XXXXXX).

Why we do it

- **Remote Access Discovery:** Gateway OS operates headless (without a monitor or keyboard). Finding the assigned IP address is mandatory to open the web management interface (LuCI) or establish an SSH terminal connection from an administration computer.

Procedure

Option A: Ethernet (Recommended)

1. Open your router or DHCP server administration page.
2. Check the active DHCP client list for a device named OpenWrt, ChirpStack, or RaspberryPi.
3. Note the IP address as <GATEWAY_IP>.

Option B: Wi-Fi Commissioning Access Point

If Ethernet is not available, connect your computer to the fallback wireless network:

SSID: ChirpStackAP-XXXXXX
Password: ChirpStackAP
Gateway address: 192.168.0.1

Step 6: Open LuCI and complete the first login

What this step does

Accesses OpenWrt's LuCI web administration interface via web browser (http://<GATEWAY_IP>/) and logs in for the first time using the default root account with an empty password.

Why we do it

- **Initial Administrative Access:** Opens the management GUI required to perform security hardening, set passwords, configure network interfaces, and check service status.
- **Credential Separation:** Ensures administrators do not confuse the Wi-Fi commissioning password (ChirpStackAP) with the system root account password.

Procedure

1. Open a browser and navigate to http://<GATEWAY_IP>/ (or <http://192.168.0.1/> if connected via Wi-Fi AP).
2. Accept the browser self-signed HTTPS certificate warning after confirming the IP address matches your gateway.
3. On the login screen, enter:

Field	First-login value
Username	root
Password	Leave completely empty (do not type anything)

4. Select **Login**.

[!NOTE] Do not type not set or ChirpStackAP in the password field. Clear any browser autofill values before clicking Login.

Step 7: Set and verify the root password immediately

What this step does

Configures a new, secure administrative password for the root user via LuCI (**System > Administration**), applies the setting, logs out, and verifies login using the new credentials.

Why we do it

- **Security Hardening:** Gateway OS ships with an unauthenticated/empty root password by default. Setting a strong root password immediately secures both the LuCI web interface and SSH command line against unauthorized network access.
- **Shared Identity:** On Gateway OS, the root password protects both the web interface and SSH terminal.
- **Verification:** Logging out and logging back in proves that the password was saved correctly before modifying network settings that could affect connectivity.

Procedure

1. In LuCI, navigate to **System > Administration**.
2. Under **Router Password**, enter your new strong password in both fields.
3. Select **Save & Apply**.
4. Log out of LuCI using the top-right logout button.
5. Log back in using: * **Username:** root * **Password:** <YOUR_NEW_ROOT_PASSWORD>
6. Confirm that login succeeds and the status dashboard opens.

Step 8: Configure the management network

What this step does

Sets the system hostname, time zone, NTP synchronization servers, network interfaces (Ethernet / Wi-Fi backhaul), and disables the fallback commissioning Wi-Fi Access Point (ChirpStackAP).

Why we do it

- **Time Synchronization (NTP):** Accurate UTC system time is mandatory for validating SSL/TLS certificates when connecting to MQTT brokers / Network Servers and for creating accurate LoRaWAN packet timestamps (time field).
- **Network Stability:** Setting static IP addresses or DHCP reservations ensures predictable, long-term remote connectivity for gateway management and data backhaul.
- **Remove Open Access Points:** Disabling ChirpStackAP removes an unencrypted, public wireless entry point into your gateway once commissioning is finished.

Procedure

1. Set Hostname and Time Synchronization (NTP)

1. Open **System > System**.
2. Enter a unique **Hostname** (e.g. lora-gw-01).
3. Select your local **Timezone**.

4. Under **Time Synchronization**: * Ensure **Enable NTP client** is **checked**. * Ensure **Use DHCP advertised servers** is **checked**. * Verify NTP candidates (0.openwrt.pool.ntp.org, etc.) or add your corporate NTP server.
5. Select **Save & Apply**.

2. Configure Backhaul Interface

1. Open **Network > Interfaces**.
2. Edit the management interface (lan or wan).
3. Keep **DHCP client** (with a router reservation) or set a static IP if required by your network team.
4. Select **Save & Apply**.

2a. Automated Gateway OS IP Helper (set-ip.sh)

To easily switch between **DHCP** and a **Static IP** on Gateway OS (OpenWrt) for either **Ethernet** or **Wi-Fi** backhaul without manually editing UCI configuration tables, install the automated set-ip.sh helper script.

Installation Command:

Paste this script creation block into your Gateway OS SSH terminal:

```
cat << 'EOF' > /usr/bin/set-ip.sh
#!/bin/sh
set -e

TARGET="${1:-}"

# Backward compatibility: if 1st arg is static or dhcp, default target to eth
if [ "$TARGET" = "dhcp" ] || [ "$TARGET" = "static" ]; then
    MODE="$TARGET"
    TARGET="eth"
    shift 1
else
    MODE="${2:-}"
    shift 2 2>/dev/null || true
fi

# Map target to UCI section
case "$TARGET" in
    eth|eth0|ethernet)
        UCI_NET="network.eth0"
        if ! uci get network.eth0 >/dev/null 2>&1; then
            if uci get network.wan >/dev/null 2>&1; then UCI_NET="network.wan"; else UCI_NET="network.lan";
        fi
        ;;
    wlan|wlan0|wlan|wlan0|wwan)
        UCI_NET="network.wwan"
        if ! uci get network.wwan >/dev/null 2>&1; then
            uci set network.wwan=interface
        fi
        # Ensure radio is enabled on boot
        uci set wireless.radio0.disabled='0' 2>/dev/null || true
        # Permit SSH & LuCI over Wi-Fi by assigning wwan to lan firewall zone
        uci add_list firewall.@zone[0].network='wwan' 2>/dev/null || true
        uci commit wireless
        uci commit firewall
        ;;
    wifi-join|wifi-connect)
        SSID="${1:-}"
        PASS="${2:-}"
        if [ -z "$SSID" ]; then
            echo "Usage: set-ip.sh wifi-join <SSID> [PASSWORD]"
            exit 1
        fi
    fi
fi
```

```

echo "[+] Joining Wi-Fi network '$SSID'..."
# Ensure radio is enabled on boot
uci set wireless.radio0.disabled='0' 2>/dev/null || true
uci set wireless.wifinet0=wifi-iface 2>/dev/null || true
uci set wireless.wifinet0.device='radio0'
uci set wireless.wifinet0.mode='sta'
uci set wireless.wifinet0.network='wan'
uci set wireless.wifinet0.ssid="$SSID"
if [ -n "$PASS" ]; then
    uci set wireless.wifinet0.encryption='psk2'
    uci set wireless.wifinet0.key="$PASS"
else
    uci set wireless.wifinet0.encryption='none'
fi
# Permit SSH & LuCI over Wi-Fi by assigning wan to lan firewall zone
uci add_list firewall.@zone[0].network='wan' 2>/dev/null || true
uci commit wireless
uci commit firewall
/etc/init.d/firewall restart 2>/dev/null || true
wifi reload 2>/dev/null || /etc/init.d/network restart
echo "[+] Joined Wi-Fi network '$SSID' and enabled SSH access over Wi-Fi (auto-starts on boot)."
```

```

exit 0
;;
*)
echo "Usage:"
echo "  Ethernet DHCP:      set-ip.sh eth dhcp"
echo "  Ethernet Static:    set-ip.sh eth static <IP> <NETMASK> <GATEWAY> [DNS1] [DNS2]"
echo "  Wi-Fi DHCP:         set-ip.sh wifi dhcp"
echo "  Wi-Fi Static:       set-ip.sh wifi static <IP> <NETMASK> <GATEWAY> [DNS1] [DNS2]"
echo "  Connect Wi-Fi SSID: set-ip.sh wifi-join <SSID> [PASSWORD]"
exit 1
;;
esac

if [ "$MODE" = "dhcp" ]; then
echo "[+] Configuring Gateway OS $TARGET interface ($UCI_NET) for DHCP..."
uci set ${UCI_NET}.proto='dhcp'
uci del ${UCI_NET}.ipaddr 2>/dev/null || true
uci del ${UCI_NET}.netmask 2>/dev/null || true
uci del ${UCI_NET}.gateway 2>/dev/null || true
uci del ${UCI_NET}.dns 2>/dev/null || true
uci commit network
/etc/init.d/network restart
echo "[+] Applied DHCP configuration to $TARGET ($UCI_NET)."
```

```

elif [ "$MODE" = "static" ]; then
IP="${1:-}"
NETMASK="${2:-255.255.255.0}"
GATEWAY="${3:-}"
DNS1="${4:-8.8.8.8}"
DNS2="${5:-1.1.1.1}"

if [ -z "$IP" ] || [ -z "$GATEWAY" ]; then
echo "Usage: set-ip.sh $TARGET static <IP> <NETMASK> <GATEWAY> [DNS1] [DNS2]"
echo "Example: set-ip.sh $TARGET static 192.168.1.151 255.255.255.0 192.168.1.1"
exit 1
fi

echo "[+] Configuring Gateway OS $TARGET interface ($UCI_NET) with Static IP: $IP (Gateway: $GATEWAY)..."
uci set ${UCI_NET}.proto='static'
uci set ${UCI_NET}.ipaddr="$IP"
uci set ${UCI_NET}.netmask="$NETMASK"
uci set ${UCI_NET}.gateway="$GATEWAY"
uci set ${UCI_NET}.dns="$DNS1 $DNS2"
uci commit network
/etc/init.d/network restart
echo "[+] Applied Static IP configuration to $TARGET ($UCI_NET)."
```

```

else
echo "Usage:"
echo "  Ethernet DHCP:      set-ip.sh eth dhcp"
echo "  Ethernet Static:    set-ip.sh eth static <IP> <NETMASK> <GATEWAY> [DNS1] [DNS2]"
echo "  Wi-Fi DHCP:         set-ip.sh wifi dhcp"
echo "  Wi-Fi Static:       set-ip.sh wifi static <IP> <NETMASK> <GATEWAY> [DNS1] [DNS2]"

```

```
    echo "    Connect Wi-Fi SSID:    set-ip.sh wifi-join <SSID> [PASSWORD]"
    exit 1
fi
EOF

chmod +x /usr/bin/set-ip.sh
```

Usage Examples:

- **Ethernet Options:** `bash set-ip.sh eth static 192.168.1.151 255.255.255.0 192.168.1.1 # Static Ethernet IP set-ip.sh eth dhcp # DHCP Ethernet`
- **Wi-Fi Options:** `bash set-ip.sh wifi-join "MyHomeSSID" "MyWifiPassword123" # Connect to Wi-Fi SSID set-ip.sh wifi static 192.168.1.152 255.255.255.0 192.168.1.1 # Static Wi-Fi IP set-ip.sh wifi dhcp # DHCP Wi-Fi IP`

2b. Configure Wi-Fi Client Mode (If not using Ethernet)

If you plan to run the gateway using Wi-Fi backhaul (connecting wirelessly to a local router instead of Ethernet):

1. Keep ChirpStackAP enabled while connected to 192.168.0.1.
2. Open **Network > Wireless**.
3. Select **Scan** on the wireless radio interface (radio0 or radio1).
4. Find your local Wi-Fi router network SSID and select **Join Network**.
5. Enter your Wi-Fi network password (WPA passphrase) and select **Submit > Save & Apply**.
6. Connect your administration computer to that same local Wi-Fi router network.
7. Check your router's DHCP client list to find the gateway's new IP address.
8. Verify you can access LuCI over the new network connection (`http://<NEW_GATEWAY_IP>/`).
9. **Only after verifying access over the new IP address**, proceed to disable ChirpStackAP.

3. Disable Commissioning Access Point

[!WARNING] **DO NOT disable the ChirpStackAP Access Point if you are currently connected through it AND do not have Ethernet plugged in!** Disabling the AP without Ethernet or an active Wi-Fi Client connection to a local router will immediately shut off wireless radio management, **locking you out of the gateway**. Only disable ChirpStackAP after you have configured Wi-Fi Client mode or connected via Ethernet, and verified you can access LuCI over the new network connection.

1. Open **Network > Wireless**.
2. Locate ChirpStackAP.
3. Select **Disable**.
4. Select **Save & Apply**.

Step 9: Verify SSH access

What this step does

Tests secure remote terminal access from your administration workstation to the gateway via SSH (`ssh root@<GATEWAY_IP>`) using the root password created in Step 7.

Why we do it

- **Command-Line Administration & Troubleshooting:** Confirms direct terminal access for inspecting low-level system logs, editing configuration files, monitoring system daemons, and running maintenance scripts.

Procedure

1. On your workstation terminal, run:

```
ssh root@<GATEWAY_IP>
```

2. Accept the SSH host key fingerprint prompt.
3. Enter the root password set in Step 7.
4. Confirm that the OpenWrt command prompt appears (root@lora-gw-01:~#).
5. (Optional) Install your SSH public key for key-based authentication:

```
ssh-copy-id root@<GATEWAY_IP>
```

Step 10: Verify the running image

What this step does

Executes diagnostic commands over SSH (cat /etc/os-release, uname -a, date -u, ip addr, monit status) to verify system OS details, time sync, network routing, and daemon status.

Why we do it

- **Operational Health Check:** Validates that the system is running the expected Gateway OS Base image, network routing and NTP synchronization are functioning properly, and critical gateway daemons (concentrator, monit) are healthy without crash loops.

Procedure

Run these commands in your SSH terminal:

```
cat /etc/os-release
uname -a
date -u
ip addr
ip route
ps w | grep chirpstack
```

Verify that: - /etc/os-release shows ChirpStack Gateway OS Base. - date -u shows the current UTC date and time (confirming NTP sync). - System process list (ps w) shows active gateway services.

Step 11: Create the first configuration backup

What this step does

Generates and downloads a compressed system backup archive (.tar.gz) from LuCI (**System > Backup / Flash Firmware**) to your administration workstation.

Why we do it

- **Disaster Recovery:** Saves a clean baseline snapshot of system configurations, network settings, and credentials off-device. In the event of SD card failure or data corruption, the gateway can be restored quickly without repeating the entire manual setup procedure.

Procedure

1. In LuCI, open **System > Backup / Flash Firmware**.
2. Under **Backup**, select **Generate archive**.
3. Save the downloaded .tar.gz file in a secure backup folder on your workstation.

Troubleshooting

The gateway does not appear in the DHCP list

- Confirm Ethernet link LEDs are lit on the Raspberry Pi port.
- Wait at least 3-5 minutes for first-boot partition expansion to finish.
- Connect to ChirpStackAP-XXXXXX Wi-Fi if available.

The browser cannot open LuCI

- Ensure your workstation is on the same network subnet as the gateway.
- Clear browser cache or use an Incognito window.
- Verify SSH access to test if the OS is alive.

System time is incorrect

- Confirm that the default gateway and DNS servers are configured under **Network > Interfaces**.
- Verify that your firewall allows outbound NTP traffic (UDP port 123).

Next Step

Continue with [03-configure-concentrator.md](#).

Supporting runbook: 02a-build-sim7600-capable-gateway-os.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 2A. Build a SIM7600-Capable ChirpStack Gateway OS Image

Use this manual only when the official ChirpStack Gateway OS Base image does not contain kernel modules required by the WaveShare SIM7600G-H.

The current Phase 11 evidence requires this branch. Gateway OS 4.12.0 on Raspberry Pi 4B runs kernel 6.6.141~ce9a7c4f21afbe9986efeaec95ee2cce-r1, while the stock OpenWrt 24.10.7 kmods repository for the same nominal kernel release requires 6.6.141~910ca1d362cc3ff4f2a1d9a4e9759bc8-r1. These ABI hashes are different. Do not force-install the stock modules into the Gateway OS kernel.

The durable repair is to build the Gateway OS image from its own pinned source/configuration with the modem support selected before the kernel and image are compiled. Direct installation on the live gateway would be acceptable only if a `kmod-*` package set built for the **exact same Gateway OS kernel ABI** were available; the stock OpenWrt packages proven in Phase 11 are not. Do not force those packages or manually copy foreign `.ko` files into `/lib/modules`.

The build host does not have to be Docker Desktop specifically. Any supported Linux environment with a working Docker engine is sufficient for the Gateway OS build. On the current workstation, Ubuntu 24.04 under WSL2 may run its own Linux Docker engine, which is simpler than depending on Docker Desktop WSL integration if Desktop is not already usable.

2A.1 Result

Design invariant: this remains ChirpStack Gateway OS **Base**, not Raspberry Pi OS, Ubuntu, or a generic writable OpenWrt install. Preserve the Gateway OS storage model: an immutable/read-only SquashFS base root filesystem with the normal writable OverlayFS persistence layer on top. The custom build must add only the required modem support and must not convert the image to a conventional fully writable root filesystem, replace the Base profile with Gateway OS Full, or bypass the normal Gateway OS sysupgrade/overlay recovery model.

Build a Raspberry Pi 4B Gateway OS **Base** image that preserves the normal ChirpStack gateway stack and additionally contains:

```
SIM7600 serial/control path
kmod-usb-serial
kmod-usb-serial-wwan
kmod-usb-serial-option

current SIM7600 data-path candidate
kmod-usb-wdm
kmod-usb-net
kmod-usb-net-qmi-wwan
uqmi
luci-proto-qmi
```

```
PPP fallback
keep the existing PPP support selected
```

The QMI packages are included because the current 1e0e:9001 composition exposes a higher vendor-specific data function while the SIMCom-aware option driver is intended for the serial functions. Do not configure QMI as the active WAN until the rebuilt gateway proves /dev/cdc-wdm* plus the associated WWAN interface actually exist.

Do not add Gateway OS Full, a local ChirpStack server, a second packet forwarder, or unrelated packages.

2A.2 Preserve the current working gateway first

Before building or flashing anything, follow [Operations 2. Gateway Backup and Recovery](#).

At minimum preserve outside the gateway:

```
official current Gateway OS 4.12.0 factory-image reference + SHA-256
Gateway OS sysupgrade configuration archive + SHA-256
current Gateway EUI = 0016c001f139a1cb
current radio plan = AS923 / topic prefix as923
current network management configuration
current Mosquitto configuration + encrypted certificate/private-key recovery bundle
current ChirpStack Concentrator/MQTT Forwarder/UDP Forwarder UCI files
```

Phase 11.0G created the current rollback artifacts on the gateway:

```
/tmp/gateway-os-backup-20260826-082144.tar.gz
SHA-256 572bfa3f45a69c5ed2ca99263988e8acf2e91ddb514f538520f62d5fb12488a1

/tmp/gateway-critical-private-20260826-082144.tar.gz
SHA-256 f74fc55480bd4edf11a745118e15eae7afac43fff9daabcd64bb20aed4757db1
```

The normal sysupgrade archive does not cover the Mosquitto certificate/data trees, so preserve **both** artifacts. The second archive is secret-bearing recovery material. Before starting the build, copy both files off the gateway over a protected channel and verify the same SHA-256 values on the receiving machine. Do not commit either archive to this repository.

For this exact Gateway OS image, Windows OpenSSH `scp` may fail with `ash: /usr/libexec/sftp-server: not found` because recent OpenSSH clients use SFTP by default and the minimal image does not ship the SFTP server helper. Use legacy SCP mode instead:

```
scp -O root@192.168.8.11:/tmp/gateway-os-backup-20260826-082144.tar.gz root@192.168.8.11:/tmp/gateway-
critical-private-20260826-082144.tar.gz <PROTECTED_DESTINATION>\
```

-O changes only the transfer protocol; it does not mutate gateway configuration.

The Phase 11.0G-2 retry passed. The protected workstation copy is:

```
C:\Users\smartagriintern\lorawan-recovery\gateway-01\20260826-082144
```

Verified destination artifacts:

```
gateway-os-backup-20260826-082144.tar.gz
size 15908 bytes
SHA-256 572bfa3f45a69c5ed2ca99263988e8acf2e91ddb514f538520f62d5fb12488a1

gateway-critical-private-20260826-082144.tar.gz
```

```
size 6188 bytes
SHA-256 f74fc55480bd4edf11a745118e15eae7afac43fff9daabcd64bb20aed4757db1
```

OFF_GATEWAY_ROLLBACK_COPY=PASS and CUSTOM_IMAGE_BUILD_ALLOWED=YES prove the configuration/private recovery artifacts are preserved. A **spare microSD card is preferred but not mandatory**. If no spare card is available, use a controlled single-card reflash path: before overwriting the current card, preserve the exact official Gateway OS 4.12.0 Raspberry Pi Base image outside the gateway, calculate and record its SHA-256, confirm the workstation can write an SD image, keep the verified configuration/private recovery bundles available, and accept planned gateway downtime during rollback. Do not overwrite the only card until those reflash prerequisites pass. Retain the gateway /tmp copies until the custom image has booted and the restore boundary has been verified.

2A.3 Build host requirements

Build on a separate administration/build machine, not on the Raspberry Pi gateway and not on a 2-GiB cloud node used by the POC.

Required:

```
Git
Docker
stable Internet access
substantial free disk space for an OpenWrt source/build tree
```

The upstream Gateway OS build is Docker-based. Building may take a long time and consume many gigabytes; that is normal.

Current build-host state - 2026-08-27

The workstation is already adequate for the build: Windows 10 Pro, 15.6 GiB RAM, 8 logical CPU threads, 126.8 GiB free on C:, WSL2, and Ubuntu 24.04.4 LTS. The Linux account smartagriintern is initialized with sudo, Git 2.43.0 is present, Internet/DNS works, and the Linux filesystem has ample space. Keep the build tree under /home/smartagriintern/src, not /mnt/c.

GNU make and the native Linux Docker toolchain are now installed and proven inside Ubuntu WSL2. GNU Make reports 4.3; Docker client/server report 29.1.3 with OSType=linux and Architecture=x86_64; Docker Compose v2 was subsequently installed and reports 2.40.3+ds1-0ubuntu1~24.04.1. Docker Desktop remains optional and is not used by the active build path.

The build workspace is /home/smartagriintern/src/chirpstack-gateway-os. An intermittent WSL DNS regression was isolated to the WSL-generated resolver 192.168.176.1: direct lookups could pass while git clone failed seconds later. A reversible temporary /etc/resolv.conf using 1.1.1.1 and 8.8.8.8 passed five repeated lookups plus git ls-remote; keep that temporary resolver only while the build may still need source/package downloads.

The repository then cloned successfully and is pinned to Gateway OS tag v4.12.0, commit 2112dbdbda48cd77ec1b82499e389abd728e84a1. Pinned submodules are chirpstack-openwrt-feed at 2a959fab57cf5a49b843ceac4b0541169e831703 and OpenWrt at b40dfac0a31695596f7c1f5f1519302ca8237f6e.

The initial Compose blocker (unknown flag: --rm) is **resolved**, not current. make init then completed the expensive feed update/install and stopped only on the final pre-environment quilt init with No series file found; the later target switch resolved the real patch path and successfully applied the Raspberry Pi target patches. The Raspberry Pi Base environment is therefore already selected. Do not rerun Docker installation, clone, feed installation, or target switching merely to reproduce earlier evidence.

The next attempted non-interactive package edit failed before changing modem package state because openwrt/scripts/config is a directory in this pinned tree. The corrected build block uses minimal .config updates

plus make `defconfig` and an eight-package verification gate. **This is historical troubleshooting, not current status:** later sections prove the build, artifact hashes, physical flash, SIM7600/QMI runtime, LTE connectivity, and the superseding 2026-09-01 flash-ready AS923+journal image. Do not resume from this intermediate point.

The earlier H-1/H-2/H-3/H-4A probes are retained as historical evidence in the Phase 11 deployment log, not as separate operator gates. From this point use one streamlined build flow:

```
Ubuntu WSL2
-> install/verify make + Linux Docker
-> clone Gateway OS v4.12.0 under Linux home
-> select bcm27xx/bcm2709 Base
-> add only SIM7600 serial/QMI packages
-> build
-> verify image/manifest/hash
-> confirm single-card rollback is reflash-ready
-> planned reflash
-> verify RAK5146 + Gateway EUI + AS923 + SIM7600
```

Stop only on a real safety/correctness failure: build dependency unavailable, wrong target/release, required package missing, build/hash verification failure, or rollback prerequisites incomplete. Do not create a new micro-phase merely for a successful read-only command.

2A.4 Pin the Gateway OS source

Clone the official repository on the build host:

```
git clone https://github.com/chirpstack/chirpstack-gateway-os.git
cd chirpstack-gateway-os
git fetch --tags --force
git checkout --detach v4.12.0
git status --short
git describe --tags --exact-match
git rev-parse HEAD
```

Record the full commit returned by `git rev-parse HEAD` with the build evidence. Do not build from `master` merely because it is newer. The running gateway is on Gateway OS 4.12.0, so reproduce that release first and change only the required modem package selection.

2A.5 Initialize the pinned OpenWrt tree

From the Gateway OS repository root:

```
make init
```

This initializes the exact OpenWrt source and feeds used by the pinned Gateway OS source tree.

Current v4.12.0 initialization compatibility note - 2026-08-27

On the current build host, `make init` completed the expensive/required initialization work successfully: both pinned submodules were checked out, all OpenWrt feeds were updated, and all feeds were installed. It then stopped only on its final command:

```
docker compose run --rm chirpstack-gateway-os quilt init
No series file found
make: *** [Makefile:17: init] Error 1
```

Do **not** repeat the feed update/install after this exact failure. The upstream Compose environment sets `QUILT_PATCHES=/workdir/conf/patches`, while the upstream `switch-env` target creates the `conf/patches` symlink only after a target environment is selected. With the current Debian `stable-slim` Quilt package, `quilt init` rejects the missing `series` file before that target link exists. Treat this as an initialization-order/tool-version compatibility issue, not a failed OpenWrt feed initialization.

For this exact state, continue directly with `make switch-env ENV=base_raspberrypi_bcm27xx_bcm2709` inside the build container. Its initial `quilt pop -a` is intentionally non-fatal, then it creates the target `.config`, `files`, and `patches` symlinks and applies the target patch series. Pass only if the target patch series exists and `quilt push -a` completes successfully. Do not create an ad-hoc empty `series`, alter the upstream Makefile, or rerun all feed installation merely to make the final `quilt init` return zero.

Then enter the Docker build environment:

```
make devshell
```

Run the remaining build commands **inside that development shell** unless the upstream build environment explicitly returns you to the host shell.

2A.6 Select the Raspberry Pi 4B Base environment

Inside the development shell:

```
make switch-env ENV=base_raspberrypi_bcm27xx_bcm2709
```

Verify the selected environment before editing package selection:

```
grep -E '^CONFIG_TARGET_bcm27xx|^CONFIG_TARGET_bcm27xx_bcm2709' openwrt/.config 2>/dev/null || \  
grep -E '^CONFIG_TARGET_bcm27xx|^CONFIG_TARGET_bcm27xx_bcm2709' .config
```

Stop if this is not the Raspberry Pi `bcm27xx/bcm2709` Base environment.

2A.7 Add only the required modem packages

The Raspberry Pi Base environment switch has already passed and applied all three target patches. On this pinned OpenWrt tree, `openwrt/scripts/config` is a **directory containing the Kconfig implementation**, not the command-line helper used by some other projects. Do not call `./scripts/config --enable ...`

For an interactive build, the upstream-supported method remains:

```
cd openwrt  
make menuconfig
```

For this reproducible Phase 11 build, use the equivalent non-interactive `.config` update: remove any existing value or not `set` line for only the required package symbols, append `=y`, then let OpenWrt make `defconfig` resolve and validate dependencies. This is the same `.config` input path used by OpenWrt's build system; `defconfig` is the authoritative normalization step.

Required packages:

```
kmod-usb-serial  
kmod-usb-serial-wwan  
kmod-usb-serial-option
```

```
kmod-usb-wdm
kmod-usb-net
kmod-usb-net-qmi-wwan
uqmi
luci-proto-qmi
```

Keep the existing Gateway OS Base selections and existing PPP support. Do not deselect RAK/ChirpStack packages and do not add unrelated modem frameworks merely because they are present in the feeds.

After the eight symbols are changed, run:

```
make defconfig
```

Then prove the intended package selection before compiling:

```
grep -E '^CONFIG_PACKAGE_(kmod-usb-serial|kmod-usb-serial-wwan|kmod-usb-serial-option|kmod-usb-wdm|kmod-usb-net|kmod-usb-net-qmi-wwan|uqmi|luci-proto-qmi)=' .config
```

All eight packages must resolve to `=y`. If `make defconfig` removes or changes any requested symbol, stop and inspect that symbol's dependencies instead of forcing the build.

2A.8 Build the image

From the environment expected by the upstream repository:

```
make
```

Do not interrupt the first build simply because it is slow.

After success, inspect the Raspberry Pi target output. The OpenWrt target artifacts normally live under:

```
openwrt/bin/targets/bcm27xx/bcm2709/
```

Locate the Base factory image and its manifest rather than guessing a filename:

```
find openwrt/bin/targets/bcm27xx/bcm2709 \
  -maxdepth 1 -type f \
  \( -name '*.img.gz' -o -name '*.wic.gz' -o -name '*.manifest' -o -name 'sha256sums' \) \
  -print
```

2A.8A Use compile time for independent rollback/readiness work

While the custom image is compiling, prepare only items that do not mutate the active source tree or the gateway SD card:

1. Download the official ChirpStack Gateway OS 4.12.0 Raspberry Pi 4B **Base factory image** from the ChirpStack Raspberry Pi download page and calculate its SHA-256 locally. Keep both the image and hash outside the gateway as the single-card rollback source.
2. Re-verify the existing off-gateway sysupgrade archive and protected Mosquitto/configuration bundle against their already-recorded SHA-256 values. Do not open or print secret-bearing file contents.
3. Confirm the workstation has an SD-card reader/writer and an imaging application ready, but do **not** remove or overwrite the gateway's only card while the build is running.

4. Record the carrier/APN information for the installed SIM, including whether the SIM requires a PIN. Do not send USB-mode-changing AT commands.
5. Keep a post-flash verification sheet ready for: Gateway OS release/kernel, RAK5146, Gateway EUI 0016c001f139a1cb, AS923, Wi-Fi management, Mosquitto persistence, MQTT Forwarder local endpoint, UDP Forwarder disabled state, SIM7600 USB binding, /dev/ttyUSB*, and QMI/PPP evidence.
6. Keep the temporary WSL DNS override in place until all source/package downloads are finished; restore the original WSL resolver only after the build no longer needs network access.

These preparations may run in parallel with compilation because they do not alter openwrt/.config, build outputs, the running gateway, or its SD card.

Current parallel-preparation result - 2026-08-27

The operator completed the non-destructive rollback preparation while the custom image continued compiling:

```
factory rollback artifact:
C:\Users\smartagriintern\lorawan-recovery\gateway-01\factory-v4.12.0\chirpstack-gateway-os-4.12.0-base-
bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
size: 27606919 bytes
SHA-256: 395e79fe041c4118e10dd4cf796aa426a565d5e733144485d8d014a8d8dbf0a6

existing sysupgrade backup re-hash: PASS
existing protected configuration/private backup re-hash: PASS

SD imaging software:
C:\Users\smartagriintern\AppData\Local\Programs\balena-etcher\balenaEtcher.exe
SD_WRITER_SOFTWARE=PASS

post-flash checklist:
C:\Users\smartagriintern\lorawan-recovery\gateway-01\factory-v4.12.0\PHASE11-POST-FLASH-CHECKLIST.txt

GATEWAY_SD_CARD_TOUCHED=NO
```

Treat these as completed prerequisites. Do not confuse SD_WRITER_SOFTWARE=PASS with proof that the physical SD reader/writer path has been exercised. Before the only production card is overwritten, still confirm the physical reader/writer path and accept the maintenance-window downtime. Carrier/APN/PIN information also remains a later LTE-commissioning input.

Build-finished status - 2026-08-27

The operator reports that the Phase 11 custom Gateway OS compilation has finished. This does **not** yet close the build branch: the final command result and generated artifacts still require verification. Preserve the existing build tree and inspect it incrementally; do not rerun make, make init, or the target switch unless evidence shows the build actually failed or an artifact is missing.

Before flashing, capture and verify the existing outputs exactly as described in section 2A.9. In particular, require all eight modem packages in the generated manifest and record SHA-256 for the factory image, manifest, and final .config. The production SD card remains untouched.

2A.9 Verify the custom image before flashing

Record SHA-256 for the exact factory image:

```
sha256sum <CUSTOM_GATEWAY_OS_FACTORY_IMAGE>
```

Inspect the generated manifest and require the modem packages:

```
grep -E '^(kmod-usb-serial|kmod-usb-serial-wwan|kmod-usb-serial-option|kmod-usb-wdm|kmod-usb-net|kmod-usb-net-qmi-wwan|uqmi|luci-proto-qmi) ' ' \
<CUSTOM_IMAGE_MANIFEST>
```

Also preserve:

```
Gateway OS tag + full source commit
OpenWrt target = bcm27xx/bcm2709
custom .config hash
factory-image filename + SHA-256
manifest filename + SHA-256
build date/time in UTC
```

Do not flash an image whose manifest does not contain the intended modem support.

2A.10 First boot on spare or controlled single-card media

A spare microSD card remains the preferred first-boot path. When only the single production card exists, do not write it until the reflash-ready rollback gate is closed. Current evidence already covers the Gateway OS 4.12.0 Base factory artifact plus local SHA-256, the verified configuration/private recovery bundles, and installed imaging software. The custom image/manifest is now verified and preserved. Still require only a confirmed physical SD reader/writer path and maintenance-window acceptance before writing the card.

Flash the custom **factory** image using the procedure in [Gateway 2](#). On the single-card path, this is the planned maintenance-window cutover and rollback means reflashing that same card with the preserved official factory image before restoring the verified backups.

Before restoring the production-like configuration, boot the custom image and verify only the platform/modem layer:

```
cat /etc/os-release
uname -r
opkg status kernel
opkg list-installed | grep -E '^kmod-usb-(serial-option|serial-wwan|net-qmi-wwan|wdm|net)'
lsmod | grep -E 'option|usb_wwan|qmi_wwan|cdc_wdm|usbserial'
ls -l /dev/ttyUSB* /dev/cdc-wdm* 2>/dev/null
ip -br link 2>/dev/null || ip link
logread | grep -Ei '1e0e|9001|option|ttyUSB|qmi|cdc-wdm|wwan' | tail -n 150
```

Expected direction for the current SIM7600G-H composition:

```
1e0e:9001 detected
serial interfaces bind to option
/dev/ttyUSB* appears
higher data function binds only to the selected data driver when supported
/dev/cdc-wdm* and a WWAN network interface appear if QMI is active
```

Do not change the modem USB PID with AT+CUSBPIDSWITCH merely to make the expected output appear.

2A.11 Restore the gateway configuration only after driver proof

Once the custom image proves the modem drivers load correctly on the selected spare-card or controlled single-card path:

1. restore the protected Gateway OS configuration archive;
2. verify Gateway EUI remains 0016c001f139a1cb;
3. verify Concentratord remains RAK5146 / AS923;

4. keep UDP Forwarder disabled;
5. verify MQTT Forwarder still targets 127.0.0.1:1883;
6. verify Mosquitto remains loopback-only and persistent;
7. preserve the existing Wi-Fi management path during LTE commissioning;
8. name the future cellular logical interface `lte`, not `wwan`, because `wwan` is already the Wi-Fi management interface in this gateway.

If spare media is available, keep the original working microSD card untouched until the rebuilt gateway passes normal-path verification. On the single-card path, do not begin the first write until the reflash-ready rollback gate above is complete; after writing, retain every off-gateway rollback artifact until normal-path verification passes.

2A.12 Pass condition

This image-build branch passes when:

```
source pinned to Gateway OS v4.12.0
Raspberry Pi Base bcm27xx/bcm2709 environment used
custom image manifest contains required SIM7600 serial drivers
stock OpenWrt foreign-ABI kmods were not installed
custom image boots on the selected spare-card or controlled single-card path
RAK5146 remains detected
SIM7600 1e0e:9001 binds to the intended drivers
/dev/ttyUSB* exists
QMI control/network interface exists if QMI is the proven data path
working gateway configuration can be restored without changing EUI/AS923 identity
rollback remains possible through the preserved official factory image plus verified off-gateway
configuration/private backups
```

After this passes, return to [Phase 11](#) for SIM registration, APN, LTE routing, QoS 1 correction, and cloud MQTT commissioning.

Post-build verifier correction - 2026-08-27

Observed verifier-only failure. The first post-build output check was launched from `/mnt/c/Users/smartagriintern`, so `git describe / git rev-parse` failed with `not a git repository` before any build artifact was inspected. This does **not** establish that the Gateway OS build failed. The operator login shell survived and no SD-card write occurred.

A second issue was found in that verifier before retry: it converted package names such as `kmod-usb-serial` to underscores when constructing `CONFIG_PACKAGE_...` symbols. OpenWrt package configuration symbols retain the package-name hyphens (for example `CONFIG_PACKAGE_kmod-usb-serial=y`). The corrected verifier must therefore `cd ~/src/chirpstack-gateway-os` explicitly and test the exact hyphenated symbols.

Do not rebuild. Resume from read-only output verification only. Do not flash until image, manifest, required package membership, and hashes are all proven.

Custom-image Stage 1 verification PASS - 2026-08-27

The corrected read-only post-build verifier completed successfully. The earlier verifier failure was only a wrong-working-directory/script issue and did not invalidate the build.

Verified source and target:

```
Gateway OS tag: v4.12.0
Gateway OS commit: 2112dbdbda48cd77ec1b82499e389abd728e84a1
OpenWrt target: bcm27xx/bcm2709
```

All eight required SIM7600 packages are present in both the final OpenWrt .config and the generated image manifest:

```
kmod-usb-serial
kmod-usb-serial-wwan
kmod-usb-serial-option
kmod-usb-wdm
kmod-usb-net
kmod-usb-net-qmi-wwan
uqmi
luci-proto-qmi
```

Generated factory image:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
size: 27672032 bytes
SHA-256: 1540958a7247e78e0e0d6791e3550f5d608865e940fc7463a5ff68229199bc3f
```

Generated manifest:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2.manifest
size: 8009 bytes
SHA-256: 49ea8900d5edccd89bafb6868f7918b71eb28c15481b67e390d69b6e7d513561
```

Final OpenWrt configuration SHA-256:

```
d17b44d29cbfc54ef40734532ee5ce900c9ff110b95c9f5107f54bbb0753cdfa
```

The target directory also contains a generated sha256sums file. Stage 1 establishes that the intended factory image exists and that its manifest contains the complete required modem package set. Before any SD-card write, still verify the generated sha256sums against the produced artifacts, preserve the custom build evidence outside the WSL build tree, restore the temporary WSL DNS after preservation, confirm the physical SD reader/writer path, and explicitly accept the maintenance-window downtime.

GATEWAY_SD_CARD_TOUCHED=NO remains authoritative.

Custom-image Stage 2 artifact-integrity PASS - 2026-08-28

The read-only artifact-integrity gate completed successfully without rebuilding, changing DNS, or touching the gateway SD card. The previously recorded factory-image, manifest, and final OpenWrt .config hashes were reverified exactly.

Generated sha256sums verification passed for every listed target artifact:

```
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz: OK
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-sysupgrade.img.gz: OK
chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2.manifest: OK
config.buildinfo: OK
feeds.buildinfo: OK
profiles.json: OK
version.buildinfo: OK
```

Authoritative hashes remain:

```
factory image SHA-256: 1540958a7247e78e0e0d6791e3550f5d608865e940fc7463a5ff68229199bc3f
manifest SHA-256: 49ea8900d5edccd89bafb6868f7918b71eb28c15481b67e390d69b6e7d513561
final openwrt/.config SHA-256: d17b44d29cbfc54ef40734532ee5ce900c9ff110b95c9f5107f54bbb0753cdfa
sha256sums SHA-256: 73ff93c061d87b9e4231776dcb420382a8e539646b64847b8e10ed4498f67e90
```

Reliable UTC mtimes recorded from filesystem epoch timestamps:

```
factory image: 2026-08-27T05:53:55Z
manifest: 2026-08-27T05:53:56Z
sha256sums: 2026-08-27T05:54:10Z
integrity verification: 2026-08-28T00:46:41Z
```

The factory image and manifest are both explicitly listed in the generated sha256sums. Stage 2 closes the custom-build artifact-integrity gate. Remaining pre-flash work is: preserve the verified custom artifacts and evidence outside WSL, verify the copied hashes, restore the temporary WSL DNS only after preservation, confirm the physical SD reader/writer path, and accept the maintenance-window downtime.

BUILD_REEXECUTED=NO, DNS_CHANGED=NO, and GATEWAY_SD_CARD_TOUCHED=NO remain authoritative for this gate.

Custom-image Stage 3 off-WSL preservation PASS - 2026-08-28

The verified custom Gateway OS output set was copied from the WSL build tree to Windows recovery storage and revalidated there without rebuilding, changing DNS, or touching the production SD card.

Preserved location:

```
C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-20260827
```

The preserved set includes the factory image, sysupgrade image, manifest, config.buildinfo, feeds.buildinfo, profiles.json, generated sha256sums, version.buildinfo, a copy of the final OpenWrt .config, and a non-secret build-identity record.

Windows-side revalidation passed exactly:

```
factory image SHA-256: 1540958a7247e78e0e0d6791e3550f5d608865e940fc7463a5ff68229199bc3f
manifest SHA-256: 49ea8900d5edccd89bafb6868f7918b71eb28c15481b67e390d69b6e7d513561
final OpenWrt .config SHA-256: d17b44d29cbfc54ef40734532ee5ce900c9ff110b95c9f5107f54bbb0753cdfa
generated sha256sums SHA-256: 73ff93c061d87b9e4231776dcb420382a8e539646b64847b8e10ed4498f67e90
```

Running sha256sum -c sha256sums against the Windows-side preserved target directory also returned OK for every generated artifact listed by OpenWrt. This closes the off-WSL custom-build preservation gate.

The DNS restoration gate is now complete. Remaining pre-flash gates are only: confirm the physical SD reader/writer path and explicitly accept the maintenance-window downtime before writing the only production card.

BUILD_REEXECUTED=NO, DNS_CHANGED=NO, and GATEWAY_SD_CARD_TOUCHED=NO remain authoritative for this preservation gate.

Custom-image Stage 4 WSL DNS restoration PASS - 2026-08-28

The temporary public-DNS workaround is no longer active. /etc/resolv.conf was already restored to the normal WSL-generated symlink /mnt/ws1/resolv.conf, whose current resolver is 192.168.176.1. Five consecutive getent ahosts github.com checks passed, so no DNS rollback was required.

Verified boundary:

```
WSL_GENERATED_RESOLVER_PRESENT=PASS
WSL_DNS_ALREADY_RESTORED=YES
WSL_RESOLV_LINK_RESTORED=ALREADY_PRESENT
WSL_RESOLV_LINK_VERIFIED=PASS
dns_lookup_1=PASS
dns_lookup_2=PASS
dns_lookup_3=PASS
```

```
dns_lookup_4=PASS
dns_lookup_5=PASS
ORIGINAL_WSL_DNS_VALIDATION=PASS
DNS_OVERRIDE_RESTORED=YES
BUILD_REEXECUTED=NO
GATEWAY_SD_CARD_TOUCHED=NO
PHASE11_WSL_DNS_RESTORE=PASS
PHASE11_WSL_DNS_RESTORE_EXIT=0
LOGIN_SHELL_SURVIVED=YES
```

The custom build, artifact integrity, off-WSL preservation, and workstation DNS cleanup gates are now closed. The physical SD reader/removable disk path is now proven. The only remaining prerequisite before shutdown/card removal is explicit maintenance-window/cutover acceptance. Do not flash until that approval is given.

Phase 11 physical SD reader preflight shell-context failure - 2026-08-28

The first physical SD reader preflight did **not** execute because the Windows PowerShell block was pasted into the Ubuntu/WSL Bash shell. Bash rejected PowerShell syntax (& {, Write-Host, Get-CimInstance, Get-PnpDevice) before any Windows disk or PnP inventory was performed. Treat this as a shell-context/verifier failure only, not as an SD-reader hardware failure.

Authoritative safety boundary from the failed attempt:

```
PHYSICAL_SD_READER_PREFLIGHT=NOT_EXECUTED
DISK_WRITE_EXECUTED=NO
DISK_INITIALIZE_EXECUTED=NO
DISK_FORMAT_EXECUTED=NO
IMAGE_WRITE_EXECUTED=NO
GATEWAY_SD_CARD_REMOVED=NO
PRODUCTION_SD_CARD_TOUCHED=NO
```

Retry by invoking Windows PowerShell explicitly from WSL or by running the block in a native Windows PowerShell prompt. Do not remove the production microSD until the empty-reader hardware baseline has passed.

Streamlined Phase 11 cutover convention - 2026-08-28

From this checkpoint onward, avoid repeating already-passed build, checksum, preservation, DNS, imaging-software, and rollback checks unless a later failure gives a concrete reason. The remaining workflow is intentionally simple:

1. confirm Windows sees the SD reader / intended removable disk;
2. accept the maintenance-window downtime;
3. flash the verified custom factory image;
4. boot the gateway and verify Gateway OS, LoRa radio, Wi-Fi management, packet forwarding, and SIM7600 driver presence;
5. continue LTE commissioning only if those checks work.

A working result is sufficient evidence. Do not add extra test layers merely to restate already-proven state.

The PowerShell reader-preflight attempt that was blocked by Windows script execution policy is a launcher issue only, not a hardware failure. `PRODUCTION_SD_CARD_TOUCHED=NO` remains authoritative.

Physical SD reader / removable disk detection PASS - 2026-08-28

Windows detected the connected SD reader/media path clearly enough for cutover:

```
Disk 1
Model: Mass Storage Device USB Device
InterfaceType: USB
```

MediaType: Removable Media
Size: 31946987520 bytes

Treat this as sufficient physical reader/disk evidence for the streamlined Phase 11 cutover. Do not repeat reader diagnostics unless the device later disappears or imaging software cannot see the card.

The production SD card has not yet been removed or written. The only remaining gate before shutdown/card removal is explicit maintenance-window/cutover acceptance. After approval, proceed directly to controlled shutdown, remove the card, re-identify the removable disk by model/size, flash the verified custom factory image, boot, and verify the gateway works.

PRODUCTION_SD_CARD_TOUCHED=NO remains authoritative at this checkpoint.

Phase 11 maintenance-window / cutover approval - 2026-08-28

The operator explicitly approved proceeding with the Phase 11 single-card cutover. Planned gateway downtime and the documented rollback path are accepted.

Current cutover sequence is intentionally streamlined:

1. gracefully power off the live gateway at 192.168.8.11;
2. remove the production microSD only after shutdown completes;
3. insert it into the already-proven Windows removable-media path and confirm the same ~31.9 GB device;
4. flash the verified custom Gateway OS factory image;
5. reinstall the card, boot, and verify LoRa, Wi-Fi management, existing MQTT path, and SIM7600 driver presence.

Do not repeat already-passed build, checksum, preservation, DNS, or reader checks unless a later step actually fails.

MAINTENANCE_WINDOW_ACCEPTED=YES CUTOVER_APPROVED=YES PRODUCTION_SD_CARD_TOUCHED=NO at this approval boundary.

Production microSD identified for cutover - 2026-08-28

Windows re-identified the inserted production microSD immediately before flashing:

Disk 1
Model: Mass Storage Device USB Device
Interface: USB
Media: Removable Media
Size: 31946987520 bytes

This matches the previously observed removable reader/media path and is sufficient for the streamlined cutover. Proceed with balenaEtcher using the preserved custom factory image under C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-20260827\target-output\. Do not select the same-named official rollback image under factory-v4.12.0.

Production microSD custom-image flash PASS - 2026-08-28

The approved Phase 11 single-card cutover has now written the verified custom ChirpStack Gateway OS factory image to the production microSD using balenaEtcher, and the operator reports the flash completed successfully. The written source was the preserved custom image under C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-20260827\target-output\, not the same-named official rollback image under factory-v4.12.0.

Because a factory image replaces the previous card contents, do not assume the prior Wi-Fi management address/configuration is already present on first boot. First boot should use Ethernet DHCP when available or the Gateway OS commissioning AP, allow first-boot filesystem expansion/reboot to finish, and prove the SIM7600 runtime driver path before restoring the verified gateway configuration/private backup.

Current cutover state:

```
CUSTOM_FACTORY_IMAGE_FLASHED=YES
BALEANAETCHER_FLASH_REPORTED_SUCCESS=YES
PRODUCTION_SD_CARD_WRITTEN=YES
ROLLBACK_IMAGE_PRESERVED=YES
NEXT=BOOT_CUSTOM_IMAGE_AND_PROVE_MODEM_DRIVERS
```

Custom-image first-boot SIM7600 runtime proof PASS - 2026-08-28

The freshly flashed custom Gateway OS booted successfully and runtime modem binding is proven. This closes the custom kernel/module objective; do not repeat build/package checks unless a later runtime regression appears.

Observed runtime evidence:

```
SIM7600 USB ID: 1e0e:9001
modules loaded: option, usb_wwan, qmi_wwan, cdc_wdm, usbserial
serial devices: /dev/ttyUSB0 through /dev/ttyUSB4
QMI control: /dev/cdc-wdm0
QMI network device: wwan0
qmi_wwan registered the modem data interface successfully
```

The custom image therefore provides both the SIMCom serial path and a working QMI-capable control/network path. The next step is to restore the verified Gateway OS configuration archive, then restore the protected Mosquitto/private bundle as required, and re-verify only the operational identity/path: RAK5146, Gateway EUI 0016c001f139a1cb, AS923, Wi-Fi management, local Mosquitto/MQTT Forwarder path, and UDP Forwarder disabled.

Normal Gateway OS configuration restore PASS - 2026-08-28

The verified gateway-os-backup-20260826-082144.tar.gz was restored successfully after the custom-image SIM7600 driver proof. The gateway returned to its working management/configuration state. Treat this restore step as PASS and do not repeat it unless a later service check fails. Next restore the protected Mosquitto/private bundle, then run one compact identity/service verification.

Mosquitto package reinstall PASS - 2026-08-28

After the custom factory-image flash and normal configuration restore, the protected /etc/mosquitto tree was restored from the verified private bundle, but the Mosquitto executable/init script was absent because the original broker packages had lived in the previous writable overlay. opkg update completed with signed OpenWrt 24.10.7 feeds, then opkg install mosquitto-ssl mosquitto-client-ssl installed Mosquitto 2.0.18-r4 and dependencies successfully.

opkg detected that the restored /etc/mosquitto/mosquitto.conf differed from the package default and preserved the restored production file, placing the package default at /etc/mosquitto/mosquitto.conf-opkg. Treat this as expected/pass behavior. Next boundary: confirm /etc/config/mosquitto uses option use_uci '0', then enable/restart Mosquitto and verify loopback-only 127.0.0.1:1883.

Mosquitto local recovery PASS / bridge TLS pending - 2026-08-28

After reinstalling mosquitto-ssl and mosquitto-client-ssl, the restored static broker configuration is active with mosquitto.owrt.use_uci=0. Mosquitto is running as mosquitto -c /etc/mosquitto/mosquitto.conf and listens

only on 127.0.0.1:1883. The local MQTT Forwarder reconnected as gateway EUI 0016c001f139a1cb, so the local packet-forwarding-to-buffer path is restored.

The two restored bridges still target the old/test endpoint lora-test-server:8883 and currently report TLS errors. Treat this as a separate outbound bridge TLS issue, not as a failure of the custom image, modem drivers, local broker, or restored gateway configuration. Do not weaken verification with `bridge_insecure true`; isolate the TLS failure with the documented `openssl s_client mTLS/hostname` test.

Restored bridge failure reclassified as network reachability - 2026-08-28

The restored Mosquitto bridges still target the lab endpoint lora-test-server:8883. The direct `openssl s_client` test failed before TLS negotiation with `BIO_connect: Host is unreachable / errno=113`. The documented lab name maps lora-test-server to 192.168.8.50. Therefore the current blocker is IP reachability to the old lab server, not CA validation, certificate hostname validation, client-certificate rejection, or the custom Gateway OS image. Do not regenerate certificates or weaken TLS. Check the restored `/etc/hosts` mapping, gateway route, and reachability of 192.168.8.50 first.

Cloud MQTT cutover decision - 2026-08-28

The restored lora-test-server:8883 bridge is now retired from the active Phase 11 path. Do not spend additional time repairing the old lab VM merely to complete the custom-image cutover. The intended production gateway path remains local MQTT Forwarder -> loopback Mosquitto buffer -> public mqtt.<DOMAIN>:8883.

The cloud public MQTT endpoint is now commissioned as smartagri-mqtt.duckdns.org:8883 through Reserved IPv4 129.212.208.168. The broker identity preserves its internal name and includes the public DuckDNS SAN; the real Gateway EUI client certificate completed strict hostname-verified mTLS, MQTT CONNECT, and an authorized command subscription. Do not point the gateway at a raw Droplet public IP.

When the physical gateway is reachable, change only the two Mosquitto bridge address lines from the retired lab endpoint to smartagri-mqtt.duckdns.org:8883, preserve the existing CA/client certificate/key and strict hostname verification, then prove buffered QoS 1 uplink delivery before making LTE the normal route. Reserved-IP reassignment/failover authority remains a separate HA acceptance item and does not block this normal-path cutover.

Post-build field LTE commissioning PASS - 2026-08-28

The custom image's modem support has now been proven beyond driver enumeration. On the physical gateway, `/dev/ttyUSB2` returned live SIM7600 AT responses, the SIM reported `+CPIN: READY`, the modem registered to DITO LTE (`+CEREG: 0,1, +COPS: ... "515 66 DITO",7`), and the DITO data APN `internet.dito.ph` was configured through OpenWrt logical interface `lte` using QMI on `/dev/cdc-wdm0`.

`ifup lte` created the dynamic `lte_4` child on kernel device `wwan0` and obtained IPv4 100.73.25.125/30 with carrier gateway 100.73.25.126. A temporary host route forced 1.1.1.1 through that LTE path and returned public ping replies with exit status 0; the temporary route was then removed. This proves that the custom Gateway OS build provides a working end-to-end SIM7600 QMI data path, not merely loadable kernel modules.

Do not duplicate the LTE commissioning commands here. The authoritative, repeatable field procedure is [Phase 11 section 11.9 - Verified SIM7600 / DITO QMI commissioning path](#).

That `peerdns=0` state was an early commissioning safeguard before the public MQTT/DNS path existed. Production has since advanced: keep `network.lte.defaultroute=0` so the health controller owns LTE route promotion, but use `network.lte.peerdns=1` so LTE can provide upstream DNS when Wi-Fi is unavailable. The image/build objective itself is complete; the reproducible overlay carries the current production policy.

Flash-ready AS923 + SIM7600 + journal release - 2026-09-01

The current accepted factory image supersedes the earlier modem-only candidate for new flashes. It is built from ChirpStack Gateway OS v4.12.0 commit 2112dbdbda48cd77ec1b82499e389abd728e84a1, OpenWrt b40dfac0a31695596f7c1f5f1519302ca8237f6e, target bcm27xx/bcm2709, and the proven DEVICE_rpi-2 profile.

```
file=chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
bytes=28900364
sha256=bafe8b97baf9353df2654b1c8b71fa53d2ff764cd264d0ed6c924dd25a5ec67d
recovery=C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-as923-journal-20260901
```

Verified contents include RAK5146/SX1302 with AS923 / as923, AS923-1 923.2/923.4 MHz channel-file entries, SIM7600 serial/QMI support, uqmi, mosquitto-ssl 2.0.18-r4, ChirpStack Concentrator 4.7.1, MQTT Forwarder 4.6.0, and gateway-evidence 0.1.0-r2. Local Mosquitto listens only on 127.0.0.1:1883; a first-boot UCI-default creates/chowns its persistent data directory before s80mosquitto. The evidence writer is boot-enabled; the uploader is installed but disabled until its HTTPS/mTLS credentials are provisioned. No evidence TLS secrets are baked into the image.

The reproducible non-secret inputs are tracked under deployment/gateway/image-overlay/. The historical full SIM7600 .config size, following its symlink, is 283853 bytes; earlier 190216-byte notes are stale. The Windows recovery copy was hashed again after transfer and matched the build artifact byte-for-byte. Physical radio/modem/real-uplink checks remain post-flash hardware acceptance.

Physical clean-overlay commissioning PASS - 2026-09-02

The accepted artifact above was flashed to production Gateway-01 and its immutable SquashFS was independently re-extracted during troubleshooting. The image itself was clean of MQTT bridge secrets/configuration, Evidence mTLS credentials, and commissioning-only smartagri-* /etc/hosts overrides. The reused microSD initially exposed old copies of those files because stale writable OverlayFS state survived outside the compact factory-image write. This was corrected with a deliberate OpenWrt jffs2reset, followed by clean reprovisioning from protected off-device identities rather than by preserving the accidental overlay.

After the reset, factory-clean state was proved before secrets were copied. Gateway-01 was then secured, configured with permanent Ethernet management 192.168.20.11/24 plus Wi-Fi backhaul, restored with the authoritative purpose-separated MQTT/Evidence identities, and configured with strict public mTLS. RAK5146 reported Gateway EUI 0016c001f139a1cb; active region/channel plan remained AS923 / as923. Two MQTT bridge sockets were established to 129.212.208.168:8883, Evidence /readyz returned ready, writer/uploader services ran, and a controlled reboot preserved the full state. ChirpStack last_seen_at advanced after the clean reprovision and again after reboot (2026-09-02 01:30:01.955894+00).

Offline inspection also showed that this already-accepted immutable release contains MQTT Forwarder qos '0', despite the intended store-and-forward contract requiring QoS 1. The current Gateway-01 clean overlay explicitly persists QoS 1 and passed the cloud/reboot checks. The repository image overlay is corrected to QoS 1 for a future deliberate rebuild; **do not rebuild the working gateway merely to change this immutable default**. Use ../automation/commission-clean-overlay.sh for future clean post-flash commissioning.

A real gateway-evidence segment/receipt after the reset remains coupled to the first real sensor uplink, because gateway statistics alone do not create a LoRaWAN evidence record. Current SIM7600 non-enumeration is a separate physical-modem condition; the commissioned Wi-Fi gateway path is healthy and is not blocked by it.

Supporting runbook: 06-verify-gateway-os.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 6. Verify Gateway OS, MQTT Buffering, and Integrity

This procedure verifies both gateway paths one layer at a time:

```
delivery -> MQTT Forwarder -> Mosquitto -> remote MQTT
evidence -> integrity journal -> hash chain -> cloud checkpoint/segment upload
```

Neither path is considered proven merely because its process is running.

A service showing as running is not enough. The checks must prove that:

- the RAK5146 is using the correct radio configuration;
- MQTT Forwarder publishes to the local broker;
- the local broker reaches the remote broker with the gateway certificate;
- uplinks remain queued during a WAN outage;
- the queue survives a gateway reboot;
- queued uplinks drain after connectivity returns;
- old downlink commands are not replayed after the outage;
- the journal records the same real uplink source independently of MQTT Forwarder;
- sequence and record/segment hashes remain continuous across normal operation and reboot;
- a WAN outage does not stop local journaling or create a false server checkpoint;
- recovery uploads missing segments and extends the last accepted off-device checkpoint;
- the server can reconcile journal evidence with the delivered remote gateway MQTT event.

Run the outage and reboot tests in staging or during an approved maintenance window.

Before you begin

Have these values ready:

```
Gateway address: <GATEWAY_IP>
Gateway EUI: <GATEWAY_EUI>
Region topic prefix: <REGION_TOPIC_PREFIX>
Remote broker: <MQTT_BROKER_FQDN>:8883
Expected queue free-space reserve: <BUFFER_FREE_SPACE_RESERVE_BYTES>
Journal implementation/version: <PINNED_JOURNAL_VERSION>
Journal storage budget: <JOURNAL_STORAGE_BUDGET>
Evidence ingest endpoint: https://<EVIDENCE_INGEST_FQDN>
```

For the real radio tests, use an approved OTAA device configured for the same region and channel plan as the gateway and ChirpStack server.

When the gateway or device has not yet been registered in ChirpStack, complete the relevant sections of [Register and test the gateway](#), then return to the traffic tests in this manual.

Step 1: Verify Gateway OS and network access

Run over SSH:

```
cat /etc/os-release
uname -a
date -u
ip addr
ip route
nslookup <MQTT_BROKER_FQDN>
monit status
ps w | grep '[m]osquitto'
```

Confirm that:

- the expected ChirpStack Gateway OS Base release is running;
- the UTC date and time are correct;
- the intended backhaul interface has an address;
- a default route exists;
- the remote broker name resolves to the expected address;
- Concentrator and MQTT Forwarder remain running;
- the local Mosquitto process is running;
- when the reviewed journal implementation has been installed, its service and uploader are healthy without repeatedly restarting.

Fix network or time problems before testing TLS. An incorrect clock can make a valid certificate appear expired or not yet valid.

For the commissioned Gateway-01 field topology, the normal production backhaul is the SIM7600 QMI interface `wwan0`; RJ45/`br-lan` at `192.168.20.11` is management-only and must not become the Internet default. After boot, allow the modem and one `lte-route-health` cycle to converge, then verify that the service is running, QMI has a current address/gateway, the metric-10 default is on `wwan0`, the production broker route resolves through `wwan0`, and UTC has synchronized. The controller intentionally retains a valid LTE default during a temporary broker/TLS failure so DNS/NTP and autonomous recovery remain possible.

```
ubus call network.interface.lte status
ubus call service list '{"name":"lte-route-health"}'
ip route show default
ip route get 129.212.208.168
date -u
logread | grep 'lte-route-health' | tail -n 20
```

On the 2026-09-15 reboot acceptance, DITO assigned a fresh PDP address, `lte-route-health` restored the metric-10 `wwan0` default automatically on its next cycle, the production MQTT mTLS probe passed, NTP corrected the clock, and the formal research-recorder preflight subsequently exited `0`. Treat a very early post-boot snapshot taken before the first successful health cycle as a startup observation, not as a routing failure; the route must appear automatically within the bounded controller cycle without a manual route command. The 2026-09-16 controller revision also distinguishes broker failure from LTE dataplane failure and performs staged autonomous recovery: bounded logical-interface restart first, then a cooldown-protected SIM7600 AT+CFUN=1,1 reset only if the

dataplane remains unavailable. Both stages were live-proven before the reboot-persistence check. The reboot-persistence check also passed: the daemon started from `S951te-route-health`, retained the repository script hash, used the 60-second boot grace instead of resetting the modem during QMI startup, then restored a fresh DITO PDP session, metric-10 default, and production mTLS without operator intervention.

Step 2: Verify the effective gateway configuration

Run:

```
logread -e chirpstack-concentratord | tail -n 100
uci show chirpstack-concentratord
uci show chirpstack-mqtt-forwarder
uci show chirpstack-udp-forwarder
```

Check the saved values against the setup:

- Concentratord uses the RAK5146 SX1302/SX1303 profile.
- The configured channel plan is the exact approved regional variant.
- The successful active SX1302/SX1303 startup log reports the expected 16-hexadecimal Gateway EUI; ignore inactive SX1301 gateway_id values.
- MQTT Forwarder uses the Concentratord backend.
- MQTT Forwarder connects to `tcp://127.0.0.1:1883`.
- MQTT Forwarder uses QoS 1 and the fixed `gw-fwd-<GATEWAY_EUI>` client ID when supported by the installed release.
- The MQTT topic prefix is `<REGION_TOPIC_PREFIX>`.
- UDP Forwarder has no enabled remote server.

Inspect both active broker files without displaying the private key:

```
grep -E
'^ (persistence|persistence_location|persistence_file|autosave_interval|max_queued_messages|max_queued_bytes|l
istener|protocol|allow_anonymous|include_dir)' \
/etc/mosquitto/mosquitto.conf

grep -E '^ (connection |address
|remote_clientid|cleansession|bridge_cafile|bridge_certfile|bridge_keyfile|bridge_insecure|topic )' \
/etc/mosquitto/conf.d/bridge.conf
```

Confirm that:

- `persistence` is enabled;
- message and byte limits are finite;
- the listener is `1883 127.0.0.1`;
- both bridge addresses use `<MQTT_BROKER_FQDN>:8883`;
- `bridge_insecure` is false;
- event and state topics go out at QoS 1;
- command topics come in at QoS 0;
- the uplink bridge uses a persistent session;
- the downlink bridge uses a clean session.

Step 3: Verify queue storage and permissions

Run:

```
df -h /etc/mosquitto/data
mount
ls -ld /etc/mosquitto/data /etc/mosquitto/certs
ls -l /etc/mosquitto/data
ls -l /etc/mosquitto/certs
```

Pass this step only when:

- /etc/mosquitto/data is on persistent writable storage;
- the path is not under /tmp or another temporary filesystem;
- the mosquitto user can write the data directory;
- the private key is not world-readable;
- current free space is greater than the configured reserve.

The mosquitto.db file can be absent before Mosquitto has saved any persistent state. It should appear or change after the outage test creates queued messages.

Step 3A: Compare the gateway with the off-device security baseline

The buffer setup manual created <GATEWAY_EUI>-mqtt-security-baseline.txt on the administration workstation. This step independently recalculates the broker-configuration hash and certificate fingerprints and compares them with that approved off-device record.

On the gateway, create a fresh comparison file:

```
umask 077
{
  printf 'gateway_eui=%s\n' '<GATEWAY_EUI>'
  printf 'captured_utc=%s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  sha256sum /etc/mosquitto/mosquitto.conf /etc/mosquitto/conf.d/bridge.conf
  openssl x509 -in /etc/mosquitto/certs/<GATEWAY_EUI>.crt -noout -fingerprint -sha256 -serial -dates
  openssl x509 -in /etc/mosquitto/certs/ca.crt -noout -fingerprint -sha256 -subject -issuer
} > /tmp/gateway-mqtt-security-current.txt
cat /tmp/gateway-mqtt-security-current.txt
```

On the administration workstation, copy the current values:

```
scp root@<GATEWAY_IP>:/tmp/gateway-mqtt-security-current.txt \
./<GATEWAY_EUI>-mqtt-security-current.txt
```

Compare everything except the intentionally changing capture timestamp:

```
grep -v '^captured_utc=' ./<GATEWAY_EUI>-mqtt-security-baseline.txt \
> /tmp/gateway-mqtt-baseline.normalized

grep -v '^captured_utc=' ./<GATEWAY_EUI>-mqtt-security-current.txt \
> /tmp/gateway-mqtt-current.normalized

diff -u \
/tmp/gateway-mqtt-baseline.normalized \
/tmp/gateway-mqtt-current.normalized
```

Pass condition: diff prints nothing and returns exit status 0.

If it prints a difference, **stop gateway acceptance** and identify the exact changed configuration line or certificate. Do not overwrite the baseline just because the current gateway is online. For an intentional configuration change or certificate rotation, first complete the relevant setup procedure and all validation tests, document the approved change, then capture a new baseline from the known-good state.

Clean up temporary comparison files after review:

```
rm -f \
  /tmp/gateway-mqtt-baseline.normalized \
  /tmp/gateway-mqtt-current.normalized \
  ./<GATEWAY_EUI>-mqtt-security-current.txt
```

On the gateway:

```
rm -f /tmp/gateway-mqtt-security-current.txt
```

This baseline detects later configuration/certificate drift. It does not prove that `mosquitto.db` message contents were never modified by a privileged attacker.

Step 4: Verify the loopback MQTT listener

Run:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

The listener must be:

```
127.0.0.1:1883
```

Fail this step when the listener is bound to `0.0.0.0`, `[::]`, a LAN address, or a WAN address. The local listener permits anonymous access only because it is restricted to loopback.

Step 5: Verify a local gateway event

Open an SSH session and subscribe to the local event topic:

```
mosquitto_sub -h 127.0.0.1 -p 1883 \
  -t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/#' -v
```

Generate one uplink from the approved test device.

A message must appear under the expected region prefix and Gateway EUI. The payload uses Protobuf and may not be readable as text. The topic and message arrival are the important checks.

Press `Ctrl+C` after the message arrives.

When no local event appears, troubleshoot Concentrator, the device channel plan, MQTT Forwarder, and the loopback broker before checking the remote TLS bridge.

Step 5A: Verify one journal record for the same real uplink

Complete this step only after the reviewed journal implementation described in [Gateway 4A](#) has been built, pinned, and installed.

Generate one fresh real uplink while observing both the local MQTT event and the journal's documented status/inspection interface.

Record, without exposing unrestricted payloads in ordinary logs:

```
Gateway EUI
journal sequence
boot ID
PHYPayload or approved PHYPayload digest
record hash
previous-record hash
current segment ID
```

Pass when:

- exactly one new journal sequence corresponds to the known uplink;
- the Gateway EUI is correct;
- the journal source is the supported Concentrator event interface, not a copy taken after Node-RED or ChirpStack;
- recomputing the versioned record hash produces the stored hash;
- the record references the preceding valid hash;
- MQTT Forwarder still produces the normal local event independently.

If the journal implementation has no safe command/API for inspecting and verifying a staging record, **stop deployment and add one to the reviewed implementation**. Do not parse undocumented binary files with an improvised production script.

Step 6: Verify the remote broker connection

Empty broker container stdout does not prove that no MQTT traffic arrived. Verify the topic directly first:

```
sudo timeout 45 docker exec mosquitto mosquitto_sub \
-h 127.0.0.1 -p 1883 \
-u chirpstack -P <CHIRPSTACK_MQTT_PASSWORD> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats' \
-v -C 1
```

A binary-looking Protobuf payload is normal. If this succeeds while ChirpStack still shows Last seen: Never, the gateway bridge path is proven and troubleshooting moves to ChirpStack region/backend configuration and exact EUI registration.

Then inspect application stdout if needed:

```
docker compose logs --since=5m --tail=200 mosquitto chirpstack
```

Confirm that:

- gw-up-<GATEWAY_EUI> connects with the gateway certificate;
- gw-down-<GATEWAY_EUI> connects with the same gateway identity;

- the broker accepts event and state writes only for this Gateway EUI;
- the broker allows command reads only for this Gateway EUI;
- ChirpStack receives the regional gateway event topic.

Also confirm in the ChirpStack web interface that the registered gateway's **last seen** time updates after the test uplink.

Verify cross-gateway isolation

When a second test Gateway EUI is available, attempt to publish or subscribe outside this gateway's allowed topic path using the first gateway certificate. The remote broker must deny the request.

Do not weaken the ACL when this test fails. Correct the certificate identity, topic path, or per-gateway ACL rule.

Step 7: Verify normal LoRaWAN traffic

Use the approved OTAA test device.

1. Power-cycle or reset the device so it performs a fresh join when appropriate.
2. Confirm that OTAA succeeds.
3. Send one uplink with a known payload or device frame counter.
4. Confirm that ChirpStack shows the expected Gateway EUI for the received uplink.
5. Confirm that the application receives one canonical record for the uplink.
6. Schedule one harmless Class A downlink.
7. Confirm that the device receives the downlink in the expected receive window.

QoS 1 can redeliver an MQTT message after a reconnect. The application or database must prevent one radio uplink from creating duplicate canonical records.

Step 8: Verify WAN-outage buffering

Disconnect only the gateway's WAN path. Keep the Raspberry Pi, RAK5146, Concentrator, MQTT Forwarder, and local Mosquitto running.

Before disconnecting WAN, run:

```
stat -c 'size=%s modified=%y' /etc/mosquitto/data/mosquitto.db 2>/dev/null || \
ls -l /etc/mosquitto/data
df -h /etc/mosquitto/data
```

Then perform the test:

1. Disconnect the gateway WAN or block only the route to the remote broker.
2. Keep an SSH session through a separate management path when possible.
3. Generate a known number of real uplinks.
4. Confirm that the local `mosquitto_sub` test still receives the gateway event topics.
5. Confirm that ChirpStack does not receive the outage uplinks yet.
6. Check `/etc/mosquitto/data/mosquitto.db` again. It should be created or increase after Mosquitto saves the queued state.
7. Confirm that free space remains above `<BUFFER_FREE_SPACE_RESERVE_BYTES>`.

8. Restore WAN connectivity.
9. Confirm that the uplink bridge reconnects.
10. Confirm that all expected uplinks reach ChirpStack and the application.
11. Confirm that the local queue stops growing after it drains.

The delivered MQTT count can include QoS 1 redelivery. Validate the application result using unique device identifiers and frame counters rather than assuming every MQTT delivery is unique.

Continuous queue growth after WAN recovery means the remote DNS, route, TLS, certificate, broker ACL, topic prefix, or ChirpStack subscription is still failing.

Step 8A: Verify the journal during WAN outage and recovery

Run this together with Step 8 so the same real test uplinks exercise both paths.

Before WAN loss, capture the latest accepted server checkpoint:

```
checkpoint gateway ID
checkpoint sequence
checkpoint record hash
checkpoint segment hash
server receipt ID/time
```

Then disconnect only WAN/backhaul and generate the known test uplinks.

During the outage pass only when:

- journal sequence continues increasing;
- the current segment continues to append/rotate within its configured budget;
- closed unuploaded segments remain local;
- the cloud checkpoint does **not** advance while the gateway cannot reach the evidence endpoint;
- Mosquitto and the journal use separate storage budgets and the OS free-space reserve remains safe.

After WAN returns, verify through the server-side procedures in [Gateway Integrity](#):

1. missing closed segments upload;
2. their first chain link extends the last accepted pre-outage anchor;
3. record and segment hashes verify;
4. the independently captured remote MQTT gateway events match the journal records;
5. a new checkpoint is accepted only after continuity is established;
6. unresolved missing evidence is reported as `evidence_gap` and contradictory evidence as `integrity_failure`.

Do not mark outage telemetry gateway-verified merely because Mosquitto successfully drained it.

Step 9: Verify queue persistence across reboot

Repeat this test while the WAN remains unavailable:

1. Generate several uplinks and confirm that the local queue grows.
2. Reboot the gateway cleanly:

```
sh reboot
```

3. Wait for Gateway OS to return.
4. Reconnect over SSH.
5. Confirm that Concentrator, MQTT Forwarder, and Mosquitto restart.
6. Confirm that `/etc/mosquitto/data/mosquitto.db` still exists and contains the queued state.
7. Generate another uplink while WAN is still unavailable.
8. Restore WAN connectivity.
9. Confirm that both the pre-reboot and post-reboot uplinks reach ChirpStack.
10. Confirm that the application retains one canonical record per real uplink.

The buffer is not persistent when messages queued before reboot disappear.

Step 9A: Verify journal continuity across reboot

Perform this during the same controlled WAN-outage reboot test when possible.

Before reboot record:

```
last journal sequence
last record hash
boot ID
open/last closed segment
```

After reboot:

1. verify the journal recovers the last valid complete record;
2. verify the boot ID changes;
3. verify the next sequence continues instead of resetting;
4. generate a fresh uplink;
5. verify its `previous_record_hash` references the last valid pre-reboot chain state;
6. restore WAN and prove the server accepts continuity across the reboot boundary.

Only an incomplete final/torn record may be discarded automatically. A previously complete record with an invalid hash is an integrity incident, not normal crash cleanup.

Step 10: Verify that stale downlinks are not replayed

The downlink bridge uses QoS 0 and a clean session so old commands are not intentionally retained.

1. Disconnect the gateway WAN.
2. Attempt one harmless test downlink through ChirpStack.
3. Allow the downlink and its receive window to expire.
4. Restore WAN connectivity.
5. Confirm that the expired downlink is not transmitted after recovery.
6. Send a new Class A downlink after the next uplink.
7. Confirm that the new downlink succeeds.

Stop and correct the downlink bridge when an expired command is transmitted after reconnecting.

Step 11: Verify queue limits without filling the SD card

Perform this only in staging.

1. Confirm the configured `max_queued_messages` and `max_queued_bytes` values.
2. Confirm the current free-space reserve.
3. Disconnect WAN.
4. Generate controlled test traffic until the configured queue limit is approached.
5. Confirm that storage use remains bounded.
6. Confirm that the filesystem does not become full or read-only.
7. Restore WAN and verify that retained messages drain.
8. Identify which messages were retained or dropped using device frame counters.

The correct behavior is bounded loss at the configured limit while preserving the gateway filesystem. An unlimited queue or a full SD card is a failed result.

Step 12: Verify backup and restore

Use a spare microSD card so the working gateway card remains untouched.

1. Flash the same approved Gateway OS Base factory image.
2. Restore the latest protected LuCI backup.
3. Confirm that the Mosquitto configuration and certificate files are present.
4. Confirm the private-key permissions.
5. Boot the restored gateway with the RAK5146 installed.
6. Confirm that the Gateway EUI and channel plan match the original gateway.
7. Confirm that both MQTT bridges authenticate.
8. Repeat one local uplink, one remote uplink, one safe downlink, and one short WAN-outage test.

A backup is not proven until it has restored a working gateway on separate media.

Troubleshooting by failing layer

Concentrator has no stable Gateway EUI

Return to [03-configure-concentrator.md](#). Check the RAK5146 profile, SPI assembly, power, and exact channel plan.

Local MQTT receives no gateway event

Check MQTT Forwarder's backend, loopback address, QoS, topic prefix, client ID, and packet filters. Do not change the remote broker yet.

Local events work but the remote broker receives nothing

Check gateway DNS, UTC time, bridge certificate files, certificate/key match, broker hostname, and remote ACL.

The remote broker receives events but ChirpStack does not update

Compare `<REGION_TOPIC_PREFIX>` with the regions enabled on the ChirpStack server. Confirm the exact Gateway EUI registration and the broker topic subscription.

The queue does not grow during an outage

Confirm that MQTT Forwarder publishes at QoS 1, the uplink bridge has `cleansession false`, persistence is enabled, and the persistence directory is writable by `mosquitto`.

The queue grows during normal connectivity

The bridge is not successfully delivering. Check route, DNS, time, TLS, certificate identity, broker ACL, and topic prefix in that order.

Queued messages disappear after reboot

Confirm that `/etc/mosquitto/data` is persistent, Mosquitto can write it, and the service stops cleanly during a normal reboot. Recheck the persistence settings and autosave interval.

An old downlink is transmitted after reconnecting

Confirm that the downlink bridge uses `cleansession true`, the command topic is `in 0`, and no second MQTT or UDP path is retaining commands.

Final acceptance

The gateway is ready for normal operation when all of these are true:

- Gateway OS Base boots with correct time and network access.
- Concentrator initializes the RAK5146 with the approved channel plan.
- The Gateway EUI remains stable.
- MQTT Forwarder publishes Protobuf messages to `127.0.0.1:1883` at QoS 1.
- Local Mosquitto listens only on loopback.
- Both remote bridges authenticate with the gateway certificate.
- The remote ACL restricts access to this Gateway EUI.
- The current broker configuration and certificate fingerprints match the approved off-device baseline.
- A real OTAA join, uplink, and safe Class A downlink succeed.
- Uplinks remain buffered during WAN loss.
- The queue survives a clean gateway reboot.
- Buffered uplinks drain after WAN recovery.
- Duplicate MQTT delivery does not create duplicate canonical application records.
- Expired downlinks are not replayed.
- Queue limits protect the SD card.
- A protected backup restores successfully to spare media.
- UDP Forwarder remains disabled.
- The journal independently records the supported Concentrator source path and record/segment hashes verify.
- Journal sequence/hash continuity survives a clean reboot.
- WAN loss preserves both the Mosquitto queue and local journal within separate finite budgets.

- Recovery uploads missing journal segments and extends the last accepted cloud checkpoint.
- Server reconciliation can distinguish verified, evidence_gap, and integrity_failure.
- Operators understand that the cloud-anchored history is stronger than a local checksum, but the software-only unanchored offline tail is not claimed to be unforgeable against full gateway root compromise.

Continue with [Gateway operations](#) for registration, routine tests, backup, recovery, and troubleshooting.

Current flash-image baseline - 2026-09-01

Use this exact factory artifact for the next clean flash:

```
file=chirpstack-gateway-os-4.12.0-base-bcm27xx-bcm2709-rpi-2-squashfs-factory.img.gz
bytes=28900364
sha256=bafe8b97baf9353df2654b1c8b71fa53d2ff764cd264d0ed6c924dd25a5ec67d
recovery=C:\Users\smartagriintern\lorawan-recovery\gateway-01\custom-v4.12.0-sim7600-as923-journal-20260901
```

Pre-flash acceptance already proved generated checksums, gzip/partition integrity, independent SquashFS extraction, RAK5146/SX1302 + AS923/as923 defaults, AS923-1 923.2/923.4 MHz channel files, SIM7600/QMI modules and uqmi, local Mosquitto, Concentrator/MQTT Forwarder, writer boot enablement, uploader default disablement, and no embedded gateway-evidence TLS secrets.

After flashing, validate only the hardware/runtime boundary: boot and writable overlay; RAK5146 Concentrator on AS923; SIM7600 tty/QMI enumeration; Mosquitto on 127.0.0.1:1883 with writable /etc/mosquitto/data; normal MQTT Forwarder enabled while UDP/mesh paths remain disabled; evidence writer running; uploader still disabled until mTLS provisioning; and one real uplink producing a journal record and reaching the telemetry path after protected bridge credentials are restored/provisioned.

Production MQTT bridge private keys/certificates and evidence-uploader mTLS material are intentionally not firmware contents. Keep them in the protected provisioning/recovery path.

Clean-overlay physical acceptance - 2026-09-02

Production Gateway-01 has now passed the post-flash hardware/runtime boundary after deliberately clearing stale writable OverlayFS state on the reused microSD. Independent extraction reconfirmed that the accepted SquashFS itself did not contain the stale MQTT/Evidence identities or commissioning /etc/hosts overrides. The gateway was reprovisioned from protected off-device credentials, rebooted, and reverified with:

Gateway EUI	0016c001f139a1cb
SX1302 model	rak_5146
region / channel plan	AS923 / as923
MQTT Forwarder	enabled, tcp://127.0.0.1:1883, qos=1
maintenance Ethernet	192.168.20.11/24, no default route
normal backhaul	Wi-Fi DHCP, default route via Wi-Fi
public MQTT	two established mTLS bridge sockets
Evidence	writer + uploader running, mTLS /readyz ready
commissioning host override	absent
ChirpStack last_seen_at	2026-09-02 01:30:01.955894+00 after reboot

Use ../automation/commission-clean-overlay.sh preflight before provisioning a future newly flashed/reused card. If production keys/bridge state appear before provisioning, stop and investigate/reset stale writable overlay rather than assuming the immutable image contains secrets.

The accepted 2026-09-01 SquashFS was also found to contain MQTT Forwarder QoS 0. Runtime commissioning corrected Gateway-01 to QoS 1 and the tracked image overlay is corrected for a future rebuild. The current gateway does not require a rebuild because its clean persisted runtime has already passed reboot and cloud verification.

Real post-reset assembled EMU-01 LoRaWAN traffic has created and uploaded gateway-evidence segments/checkpoints successfully. The gateway writer/uploader, public mTLS ingest path, checkpoint/segment receipts, and cloud trusted-decoder verification are physically proven. At the 2026-09-02 powered-off acceptance boundary, the last EMU-01 application uplink was frame counter 666 at 05:31:45.795 UTC; Gateway-01 subsequently closed and receipted segment 28, journal/open was empty, and the cloud verifier reported all 492 selected v2 events verified with no evidence failures. This closes the Gateway OS physical evidence-lineage boundary; later sensor work should re-check it only if the gateway/evidence configuration changes.

Supporting runbook: 05-troubleshooting.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Operations 5. Troubleshooting the Buffered Gateway

Troubleshoot the lowest failing layer first. Do not reflash the gateway or change the radio region as a first response. Unless a section says otherwise, run shell commands over SSH on the Raspberry Pi gateway.

Use one known device uplink while testing. A healthy result at one layer proves only that layer; for example, an MQTT connection does not prove the RAK5146 is receiving the correct RF channels.

Last seen: Never — checkpoint flow

When the gateway is registered but ChirpStack shows **Last seen: Never**, use these checkpoints in order and stop at the first failure:

1. **Radio / Concentratord** — active SX1302/SX1303 process is running, RAK5146 initializes on `/dev/spidev0.0`, and `logread -e chirpstack-concentratord | tail -n 100` reports the real Gateway ID retrieved EUI. Ignore inactive SX1301 gateway IDs.
2. **Gateway registration** — that exact 16-hex EUI is manually registered in ChirpStack. Missing from the list means not registered; present with Last seen Never means registered but no gateway event has been processed yet.
3. **MQTT Forwarder** — backend is Concentratord, server is `tcp://127.0.0.1:1883`, topic prefix matches the region, JSON is disabled, and TLS fields are empty.
4. **Local Mosquitto** — listener is only `127.0.0.1:1883`, Forwarder connects, and persistent state can be saved to `mosquitto.db`.
5. **TLS** — `openssl s_client` verifies CA, hostname, server certificate, and the gateway client certificate/key with return code 0.
6. **MQTT authentication / ACL** — a certificate-authenticated `mosquitto_pub` receives `CONNACK (0)` and `PUBACK (... RC:0)` on the gateway's own allowed topic.
7. **Two bridges** — `gw-up-<GATEWAY_EUI>` and `gw-down-<GATEWAY_EUI>` remain connected; normal keepalive `PINGRESP` messages are present.
8. **Server broker** — direct `mosquitto_sub` against the broker sees `<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats`. Binary Protobuf output is normal.
9. **ChirpStack configuration** — ChirpStack is running without TOML errors; `enabled_regions` contains the region; the active v4 region file uses `[[regions]]`, enables the MQTT backend, and has matching topic prefix/broker credentials. Keep backup TOMLs outside the mounted ChirpStack config directory.
10. **ChirpStack UI** — Last seen becomes recent. Only now move to real RF-device join/uplink testing.

1. Gateway does not boot or cannot be reached

What appears to be failing: the Raspberry Pi, Gateway OS, Ethernet/Wi-Fi path, or management addressing.

Most likely causes: inadequate power, a failed or corrupted SD card, no network link, a changed DHCP address, or the wrong image for the Raspberry Pi 4B.

From the commissioning workstation, check the router or DHCP server for the gateway lease, then test the observed address:

```
ping <GATEWAY_IP_ADDRESS>
ssh root@<GATEWAY_IP_ADDRESS>
```

A healthy gateway has a stable lease, responds on the management network, and opens LuCI or SSH. No lease usually points to power, boot media, or link state. A lease with no SSH response points to addressing, firewall, or a partially booted system.

The smallest safe fix is to verify the power supply and cable, reseal the SD card with power removed, and test the approved Gateway OS Base image on a spare card. Preserve the original card for log or configuration recovery. Verify the fix by rebooting once and confirming that the same management address is reachable again.

2. Concentrator has no stable Gateway ID

What appears to be failing: the service that owns the RAK5146 concentrator.

Run on the gateway:

```
monit status
uci show chirpstack-concentrator
logread -e chirpstack-concentrator
```

A healthy result shows the Concentrator service running, the SX1302/SX1303 chipset and RAK5146 shield selected, the confirmed channel plan, and one stable 16-hexadecimal Gateway ID.

Interpret abnormal output before changing anything:

```
wrong shield or chipset -> RAK5146 profile mismatch
SPI or reset error      -> HAT seating, power, reset mapping, or hardware fault
calibration loop        -> frequency variant, clock, profile, or hardware problem
Gateway ID changes      -> unstable concentrator identity or hardware state
```

For a profile mismatch, correct only the observed chipset, shield, or channel-plan value in **ChirpStack > Concentrator**. For SPI, reset, or hardware errors, power down before reseating the HAT. Do not repeatedly change the Gateway EUI in ChirpStack. Verify the fix by rebooting and confirming the same Gateway ID appears without restart or calibration loops.

3. MQTT Forwarder cannot publish to the local broker

What appears to be failing: the gateway-local hop from MQTT Forwarder to Mosquitto.

Run on the gateway:

```
uci show chirpstack-mqtt-forwarder
logread -e chirpstack-mqtt-forwarder
```

```
logread -e mosquitto
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Healthy configuration uses `tcp://127.0.0.1:1883`, Protobuf, backend concentrator, QoS 1, and a Mosquitto listener bound only to `127.0.0.1:1883`.

Abnormal results mean:

```
connection refused    -> Mosquitto is stopped, invalid, or listening on another port
WAN broker configured -> MQTT Forwarder bypasses the persistent local buffer
QoS is not 1          -> the local broker does not provide the intended durable acknowledgement
0.0.0.0 or LAN listener-> the local broker is exposed beyond the gateway
```

Validate the Mosquitto configuration before restarting it, then correct only the local address, listener, or QoS setting that differs. Verify with one real uplink: MQTT Forwarder logs should stop reporting connection errors and Mosquitto should receive the regional gateway event topic.

4. Local broker receives messages but the remote bridge is offline

What appears to be failing: DNS, route/4G, system time, TLS identity, broker ACLs, or the remote MQTT service.

Run on the gateway:

```
logread -e mosquitto
nslookup <MQTT_BROKER_FQDN>
ip route
date -u
df -h /etc/mosquitto/data
ls -l /etc/mosquitto/data/mosquitto.db
```

Healthy output shows the broker name resolving to the intended endpoint, a usable default route, correct UTC time, sufficient free space, a persistent `mosquitto.db`, and successful bridge authentication for both `gw-up-<GATEWAY_EUI>` and `gw-down-<GATEWAY_EUI>`.

Use the first matching failure:

```
DNS or route failure -> WAN path unavailable; repair DNS, APN, or route and leave the queue running
unknown CA or hostname -> wrong trust file or certificate SAN; install the intended CA and hostname
client rejected -> expired/wrong certificate, EKU, CN, or private-key mismatch
not authorized -> certificate identity, Gateway EUI, region prefix, or ACL mismatch
queue grows -> remote service is unavailable or draining more slowly than new traffic arrives
queue limit reached -> the outage exceeded finite capacity and some delivery may be lost
```

Never disable certificate verification to make the bridge connect. Correct the endpoint, trust, certificate/key pair, or exact topic ACL.

Isolate the connection in three layers:

```
# A. TCP only
nc -w 3 <MQTT_BROKER_FQDN> 8883 </dev/null
echo $?

# B. TLS, hostname, CA, and client-certificate verification
openssl s_client \
  -connect <MQTT_BROKER_FQDN>:8883 \
  -servername <MQTT_BROKER_FQDN> \
  -verify_hostname <MQTT_BROKER_FQDN> \
  -verify_return_error \
  -CAfile /etc/mosquitto/certs/ca.crt \
  -cert /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
```

```
-key /etc/mosquitto/certs/<GATEWAY_EUI>.key \
</dev/null

# C. MQTT authentication + write ACL
mosquitto_pub \
-h <MQTT_BROKER_FQDN> -p 8883 \
--cafile /etc/mosquitto/certs/ca.crt \
--cert /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
--key /etc/mosquitto/certs/<GATEWAY_EUI>.key \
-i gw-cert-test-<GATEWAY_EUI> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/state/test' \
-m 'mtls-auth-test' -q 1 -d
```

Interpret them separately: TCP exit 0 proves only route/port; TLS must report verification return code 0; MQTT CONNACK (0) proves authentication and PUBACK (... RC:0) proves the ACL allowed publication. Verify the final bridge fix by observing reconnect, queue drain without deleting mosquitto.db, and a new gateway event at the remote broker.

5. Remote broker receives gateway events but ChirpStack does not

What appears to be failing: the server-side MQTT-to-ChirpStack path.

Run this isolation test on the application server before relying on container stdout logs:

```
sudo timeout 45 docker exec mosquitto mosquitto_sub \
-h 127.0.0.1 -p 1883 \
-u chirpstack -P <CHIRPSTACK_MQTT_PASSWORD> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats' \
-v -C 1
```

A received Protobuf packet proves the gateway bridge reached the server broker even when `docker logs mosquitto --since 5m` is empty. Broker topic delivery and application stdout logging are different evidence.

Then confirm the active ChirpStack region MQTT backend subscribes to the same case-sensitive region prefix emitted by Gateway OS and uses the intended broker identity. A healthy result shows ChirpStack updating the registered Gateway EUI's last-seen time. If the broker has the topic but ChirpStack remains silent, check ChirpStack process health, TOML parse errors, `enabled_regions`, `v4 [[regions]]`, explicit MQTT backend enablement, topic prefix, credentials, and Protobuf handling.

Make the smallest correction to the ChirpStack region configuration or broker ACL, restart only the affected service, and verify with a fresh uplink. Do not add ChirpStack Gateway Bridge; this architecture sends Concentrator Protobuf topics directly to ChirpStack's MQTT region backend.

6. Duplicate application or telemetry rows

What appears to be failing: downstream idempotency, not gateway transport.

QoS 1 and reconnects can legitimately redeliver the same event. Check that Node-RED uses ChirpStack `deduplicationId` or the documented stable fallback event key and that the TimescaleDB unique indexes exist.

A healthy replay leaves one uplink row and one row per normalized metric for the event key. More than one row means the application or database duplicate guard is missing or uses an unstable key. Fix the event-key or unique-index path and replay the same sanitized event again. Do not reduce gateway QoS to hide duplicate storage.

7. Buffered messages disappear after reboot

What appears to be failing: Mosquitto persistence or the selected storage path.

Run on the gateway before and after a controlled reboot:

```
df -h /etc/mosquitto/data
mount | grep -E ' /tmp | /etc/mosquitto/data'
ls -l /etc/mosquitto/data/mosquitto.db
logread -e mosquitto
```

Healthy behavior keeps the data path on persistent storage, saves a non-empty database with the intended owner, and reopens the queue after restart. A missing database, tmpfs path, ownership error, or repeated persistence warning means the queue is not durable.

Correct the path, permissions, or persistence setting without deleting the only queue database. Verify with the documented WAN-loss, reboot, reconnect, and drain test. Do not claim durable buffering until a real event survives that sequence.

8. A stale downlink transmits after recovery

What appears to be failing: downlink session cleanup or an unintended broker queue.

Confirm the downlink bridge uses `cleansession true` and the command topic direction is `in 0`. Check that no second client session, retained message, or automation path is publishing the old command.

Healthy behavior drops commands whose Class A receive window has passed. Any stale transmission means automation must remain disabled. Remove the unintended persistent session or retained command, reconnect, then test with a harmless device command whose receive window and result can be observed.

9. RF uplinks or downlinks fail while MQTT is connected

What appears to be failing: the radio, device registration, or LoRaWAN scheduling layer.

Compare the antenna band and connection, RAK5146 frequency variant, Concentrator channel plan, ChirpStack region, device profile, device regional variant, OTAA root-key reference, frame counters, class, receive windows, gateway time, FPort, and legal transmit limits.

Healthy commissioning shows a JoinRequest, JoinAccept, a later data uplink, and—when required—a safe downlink in the expected receive window. A connected MQTT bridge with no RF event means backhaul is healthy but radio or device settings are not. Correct the first mismatched region, profile, key reference, or antenna condition and repeat one join/uplink test.

10. UDP Forwarder traffic appears

What appears to be failing: the supported architecture has been bypassed.

Run on the gateway:

```
uci show chirpstack-udp-forwarder
monit status
```

Healthy output has no UDP server configured and the UDP Forwarder disabled. Remove only the unintended UDP server entries, disable the service, and verify that MQTT Forwarder continues delivering a real uplink through the local broker.

11. Integrity journal does not advance

What appears to be failing: the independent Concentratord evidence path, not necessarily MQTT delivery.

Check the reviewed journal service status, its configured read-only Concentratord interface, persistent evidence directory, current sequence, open segment, and journal-specific logs/metrics using the implementation's documented diagnostics.

Healthy behavior shows a new sequence for a known real uplink while MQTT Forwarder independently publishes the corresponding local event.

Interpret the failure by layer:

```
Concentratord healthy, MQTT healthy, journal stopped
-> journal implementation/service/permission fault

journal process running, sequence unchanged
-> source subscription/interface or filter fault

sequence advances, writes fail
-> evidence storage/permissions/free-space fault
```

Do not point the journal at Mosquitto merely to make its counter move. Repair the independent source path and mark the unjournaled interval as an evidence gap.

12. Journal hash or sequence continuity fails

A complete record with an invalid hash, a broken previous-hash link, an unexplained sequence reset, or a segment-hash conflict is not routine log corruption.

Preserve the files and current storage image before attempting repair. Compare against the latest server checkpoint and the server's independently captured gateway MQTT events. Do not renumber records, recalculate a replacement chain in place, or delete the conflicting checkpoint.

Use:

```
missing required object with no contradictory bytes -> evidence_gap
proven hash/payload/rollback conflict               -> integrity_failure
```

13. Cloud checkpoint is stale but MQTT is healthy

What appears to be failing: the evidence uploader/authentication/API path.

Check independently:

```
DNS/route to evidence endpoint
TLS hostname and CA
per-gateway upload identity
latest local closed segment
unuploaded segment count/bytes
```

```
latest accepted server checkpoint age
API rejection/conflict logs
```

Do not treat healthy MQTT as proof that evidence upload works. Restore the uploader and verify the next accepted checkpoint extends the previous server anchor.

14. Journal and remote MQTT payload disagree

Freeze v2 evidence promotion for the affected lineage. Preserve:

```
journal segment/object
remote MQTT captured event/object
Gateway EUI
PHYPayload digests
uplink/context identifiers
timestamps
forwarder/broker versions and logs
```

A payload conflict after the Concentrator split can indicate journal corruption, MQTT Forwarder/delivery-path alteration, wrong correlation, or compromise. Do not let Node-RED choose which copy is authoritative. Use the server verifier's deterministic matching contract.

15. Gateway evidence storage approaches its limit

Restore the evidence upload path before the emergency OS reserve is threatened. Do not delete the oldest unuploaded segment simply to clear the alarm.

If retention is genuinely exhausted, preserve the last valid sequence/hash and report the affected interval as `evidence_gap`. Mosquitto may still deliver telemetry; that does not restore missing evidence.

16. A Gateway OS or package upgrade causes a regression

Preserve the failed image reference, Mosquitto data and configuration, certificates, UCI configuration, and relevant logs. Do not overwrite the only known-good SD card.

Restore the previously tested image and configuration on spare media, boot it with the same RAK5146, and repeat Gateway ID, region, local publish, journal source/sequence/hash checks, cloud checkpoint/segment upload, WAN outage, reboot, queue drain, server reconciliation, OTAA uplink, and safe downlink checks. A successful rollback must restore behavior, not merely start the services.

Canonical source: [deployment/technology-guides/01-gateway-os-openwrt.md](#). For any mutable production-status statement, check the dated baseline and verified live state.